Highlights

SustainSched-MPC: Risk-Constrained Power-, Carbon-, and Thermal-Aware Scheduling for Sustainable Distributed Computing

Hemant Palivela,Afreen Arif

- **Gap:** No prior framework jointly enforces energy, carbon, thermal, and probabilistic SLA constraints in geo-distributed scheduling
- **Novelty:** Chance-constrained MILP with RC thermal dynamics solved online every 5 min via Lagrangian decomposition (<2.3 s/decision)
- **Scale:** 30-day, 0.55 M-job trace (18,240/day) on 3-region cluster with real Electricity Maps MEF carbon signals and IPMI-calibrated power models
- **Impact:** –20.5% energy, –32.1% CO_2e, –48.8% hotspot time vs. EDF; –12.9% CO_2e and –56.3% SLA miss vs. carbon-only MPC
- **Decision support:** Single risk-budget knob sweeps a convex Pareto frontier; three deployment recipes quantified for practitioners

SustainSched-MPC: Risk-Constrained Power-, Carbon-, and Thermal-Aware Scheduling for Sustainable Distributed Computing

Hemant Palivela[a,*], Afreen Arif[a]

[a]Amity University Dubai, Dubai International Academic City, Dubai, , Dubai, United Arab Emirates

ARTICLE INFO

Keywords:
sustainable computing
power-aware scheduling
thermal-aware management
green data centers
model predictive control
carbon-aware orchestration
energy optimization

ABSTRACT

Problem. Data centers now consume 200–250 TWh/year globally, a figure projected to double by 2030. Schedulers that optimize energy, carbon, and thermal safety in isolation create compensating degradations—a carbon-focused scheduler that concentrates workload on low-intensity slots generates thermal hotspots; a thermal-cap policy forfeits carbon savings; a power optimizer schedules into carbon-expensive peaks. No production-ready framework jointly controls all three under explicit, probabilistic service-level guarantees.

Novelty. We present **SustainSched-MPC**: the first receding-horizon, chance-constrained scheduler that *simultaneously* optimizes energy consumption, operational CO_2e, thermal stress, and SLA deadline reliability for geo-distributed computing. A unified MILP is solved every 5 min over a 60-min prediction horizon via Lagrangian region decomposition (<2.3 s wall-clock). Per-class deadline chance constraints—reformulated from stochastic runtime forecasts via calibrated uncertainty quantiles—bound SLA miss probability without sacrificing carbon reduction.

Evaluation scale. We conduct a fully reproducible 30-day, 8,640-interval trace-driven evaluation on a 24-rack (960-server) geo-distributed cluster spanning 3 regions with real carbon-intensity time-series (Electricity Maps MEF, January 2024), calibrated IPMI power and RC thermal models, and a mixed batch/latency-critical workload of 0.55 M jobs (18,240/day × 30 days). Five schedulers are compared across seven metrics over 5 fixed random seeds (25 total runs).

Results. Versus deadline-first (EDF): **–20.5%** energy, **–32.1%** CO_2e, **–48.8%** thermal hotspot time, SLA misses 0.49% (+0.07 pp). Versus carbon-only MPC: **–12.9%** CO_2e *and* **–56.3%** SLA violations simultaneously. Ablation confirms each component (thermal term, carbon term, chance constraints, DVFS, migration) is independently necessary. A single risk-budget multiplier $\alpha \in [0.5, 3.0]$ (scaling per-class budgets $\epsilon_s(\alpha) = \alpha \cdot \epsilon_s^0$) sweeps a convex Pareto frontier from 24.8% to 36.9% CO_2e reduction.

Decision support. Three deployment configurations (reliability-priority, balanced, carbon-priority) are quantified with concrete parameter settings. At 1,000-node scale, the balanced mode avoids **2,281 tCO_2e/year** and saves **$398,000/year** in energy cost, providing a clear business case alongside the environmental benefit.

1. Introduction

Data centers and distributed computing platforms now account for approximately 200–250 TWh of global electricity consumption annually, a figure projected to double by 2030 as artificial intelligence workloads, cloud services, and high-performance computing (HPC) continue to expand [37, 36, 26, 22]. The carbon footprint of this consumption depends critically on the regional grid energy mix: facilities operating in fossil-fuel-dominated grids can emit tens of thousands of tonnes of CO_2 equivalent per year, while those co-located with renewable generation emit only a fraction for identical computational throughput. Exploiting this spatiotemporal variability—shifting computation to low-carbon windows and low-intensity regions—is the central premise of carbon-aware computing [36, 7, 38, 35, 21, 18]. Yet carbon-aware scheduling cannot operate in isolation: its decisions feed back directly into power draw, node temperatures, and service-level reliability, creating a three-way coupling that single-objective schedulers are structurally ill-equipped to handle.

Thermal management represents an independent but deeply coupled operational constraint. Sustained node temperatures above 80–85°C accelerate component wear, increase failure rates, and trigger hardware-level frequency throttling that degrades throughput in ways that are difficult to predict at scheduling time. Cooling infrastructure—precision air conditioning, liquid cooling, and economizers—typically accounts for 30–40% of total data center energy expenditure, and this fraction rises superlinearly with average rack temperature [9, 3]. A scheduling policy that improves energy efficiency or reduces carbon emissions by concentrating workload on fewer nodes may inadvertently generate thermal hotspots that offset those gains through elevated cooling demand, reduced hardware longevity, and triggered throttling.

Power-performance management through dynamic voltage and frequency scaling (DVFS), server sleep-state transitions, and workload consolidation constitutes the third dimension of the sustainability challenge [1, 8, 45, 10]. These mechanisms interact non-trivially with thermal behavior: lower operating frequencies reduce heat generation but also reduce per-slot computational throughput, affecting the pace of workload completion and therefore the risk of missing service deadlines. Service-level agreements (SLAs)—operator commitments to complete jobs within specified deadlines

*Corresponding author
hemant.palivela@amityuniversity.ae (H. Palivela); afreen.arif@amityuniversity.ae (A. Arif)
ORCID(s):

with a defined reliability—are the ultimate accountability boundary for multi-tenant distributed platforms, and no sustainable scheduling framework can ignore them.

The fundamental difficulty is that power, thermal, and carbon objectives are *jointly determined* by the same workload placement and timing decisions: optimizing any single dimension while treating the other two as soft penalties or ad-hoc post-processing leaves the system vulnerable to compensating degradations. A carbon-focused scheduler that concentrates workload into low-intensity windows raises utilization and temperatures during those periods, potentially triggering throttling, hotspot violations, and SLA failures [36, 39]. A thermal-cap policy that limits server utilization to suppress hotspot formation may forgo carbon savings achievable through coordinated spatial workload migration [3, 4]. Power-only optimizers regularly improve energy figures while incidentally scheduling in carbon-expensive periods [1, 6]. This isolation-of-objectives pattern is pervasive in the literature and is quantitatively confirmed by the ablation analysis presented in Section 9.

A secondary challenge is *online adaptability under forecast uncertainty*. Carbon-intensity signals carry prediction errors that grow with horizon length; workload arrival is stochastic; and node temperatures evolve continuously with ambient conditions, cooling dynamics, and incoming job streams. Static offline plans cannot respond to intra-day deviations, and reactive heuristics lack the look-ahead to avoid constraint violations before they materialize. Model predictive control (MPC), through its receding-horizon re-optimization structure [15], provides a principled mechanism for online adaptation: at each control interval the scheduler solves a constrained optimization over a finite prediction horizon, commits only the immediately actionable decision, and then re-solves at the next step with refreshed state information. This structure has demonstrated strong practical performance in microgrid energy management [40, 41] and chip-level thermal regulation [42, 43], but its extension to joint power-carbon-thermal scheduling in geo-distributed cloud platforms operating under explicit probabilistic SLA constraints remains unrealized. Recent work on sustainable governance of distributed continuum systems [17] and holistic sustainable computing frameworks [33] further underscore the gap between broad sustainability goals and the fine-grained scheduling mechanisms needed to achieve them.

Chance-constrained programming [16] provides the mathematical tool to embed SLA reliability directly into the optimization: by requiring that the probability of a deadline violation remain below an operator-specified tolerance ϵ_s, the formulation converts stochastic runtime forecasts into deterministic capacity-margin requirements that are tractable to solve online. Existing carbon-aware schedulers address reliability through risk-aware heuristics [36] or distributionally robust capacity curves [39], but neither framework integrates this reliability control with thermal and power co-optimization inside a single receding-horizon optimizer.

This paper presents **SustainSched-MPC**, a risk-constrained, power-, carbon-, and thermal-aware scheduling framework for sustainable distributed computing. The framework integrates queue dynamics, multi-region resource constraints, DVFS-parameterized power models, first-order RC thermal dynamics, and carbon-intensity forecasts into a unified chance-constrained multi-objective optimization solved online every 5 minutes via a rolling 12-slot prediction horizon. The main contributions are:

1. **Unified multi-objective formulation:** A joint optimization over energy consumption, carbon emissions, thermal stress, and SLA penalty with explicit per-class chance constraints on deadline violation probability, grounded in first-order RC thermal dynamics and DVFS-parameterized power curves that are calibrated from infrastructure telemetry.
2. **SustainSched-MPC framework:** An online rolling-horizon scheduler that decomposes the problem by region with shared SLA risk budgets, uses warm-start acceleration to bound solve latency to 2.3 s per decision, and adaptively calibrates uncertainty reserves from realized SLA and thermal outcomes in a closed-loop feedback structure.
3. **Feasibility and complexity analysis:** Sufficient conditions for constraint feasibility derived from risk-adjusted capacity envelopes, validity of the chance-constraint reformulation under calibrated uncertainty quantiles, and an $O(nmh)$ binary complexity characterization that enables practical runtime deployment within the 5-minute control interval.
4. **Reproducible 30-day evaluation:** A systematic trace-driven evaluation on a 24-node geo-distributed cluster model demonstrating 20.5% energy reduction, 32.1% CO_2e reduction, and 48.8% thermal hotspot reduction relative to deadline-first scheduling, while maintaining SLA misses at 0.49%, with full baseline-wise quantitative comparisons, sensitivity analysis across risk budgets, and ablation of individual framework components.

The remainder of this paper is organized as follows. Section 2 reviews related work and positions SustainSched-MPC within the literature through a structured comparison table. Sections 3–5 define the system, power-thermal-carbon, and optimization models. Section 6 presents the proposed framework. Section 7 provides theoretical analysis. Section 8 details the experimental setup. Section 9 reports results and discussion. Section 10 concludes.

2. Related Work

2.1. Power- and Energy-Aware Scheduling

Energy-aware scheduling is the most extensively researched dimension of sustainable distributed computing, with a substantial body of work spanning cloud, HPC, and cluster environments. Beloglazov et al. [1] established foundational heuristics for VM consolidation and live migration

in cloud data centers, demonstrating that proactively exploiting host idle-power states through workload bin-packing yields substantial energy savings; the framework does not, however, account for carbon-intensity signals or the thermal consequences of concentrating workload on fewer, hotter servers. Kliazovich et al. [2] complemented this perspective by modeling network energy as a first-class allocation objective, showing that neglecting interconnect costs leads to suboptimal total energy profiles even when compute utilization is individually optimized. Gai et al. [30] demonstrated resource management in sustainable cyber-physical systems using heterogeneous cloud computing, showing that heterogeneity-aware allocation substantially reduces energy overhead in mixed infrastructure. Dorronsoro et al. [6] applied hierarchical evolutionary algorithms to balance energy and makespan for independent task scheduling in clusters, while Li [10] characterized energy-time tradeoffs under irregular DVFS curves where discrete power levels do not scale uniformly with frequency. Wang et al. [11] designed energy-agile scheduling primitives for parallel HPC applications that respond to real-time power-budget signals, but without carbon or thermal integration.

More recent contributions have broadened the scope of energy-aware scheduling. Kocot et al. [45] conducted a comprehensive survey of energy-aware scheduling in modern HPC systems, systematically cataloguing DVFS, power capping, consolidation, and machine learning-based approaches; the survey explicitly identifies joint multi-objective treatment as rare and carbon awareness as largely absent from the field. Chandio et al. [12] targeted VM scheduling for HPC workloads in cloud data centers with energy-efficient placement heuristics. Borghesi et al. [8] demonstrated lightweight scheduling-based power capping in production HPC systems without requiring hardware instrumentation. Cao et al. [46] proposed an energy-aware intelligent scheduling algorithm (EIS) for deadline-constrained cloud workflows, minimizing idle gaps and distributing slack time to reduce energy without violating deadlines; SLA management is heuristic and carbon signals are absent. Santhosh et al. [20] proposed a decentralized, energy-efficient cloud architecture to reduce idle resource consumption, while De et al. [23] optimized VM allocation across decentralized peer-to-peer nodes to improve data center sustainability. Abbas et al. [27] integrated renewable energy sources directly into a green computing framework to profile and improve energy efficiency across heterogeneous nodes. Sharma et al. [25] addressed sustainable energy-efficient workflow scheduling in geo-distributed cloud data centers, demonstrating that joint classification and placement of workflows reduces both energy and carbon footprint; however, thermal dynamics and formal SLA risk bounds remain unaddressed. Wang et al. [31] extended multi-objective task processing at the edge to include energy-efficiency objectives with deadline constraints, further extending the geographic scope of energy-aware scheduling. Across this body of work, energy is treated as the dominant sustainability metric, with carbon intensity and thermal dynamics either ignored or approximated by static proxies.

2.2. Thermal-Aware Resource Management

Thermal management addresses the coupled problems of hotspot prevention and cooling overhead reduction. Banerjee et al. [3] performed one of the earliest formal analyses co-optimizing workload placement with cooling-system awareness in HPC data centers, demonstrating that rack-level placement decisions directly determine cooling costs and that ignoring this coupling leads to avoidable thermal penalties. Sun et al. [4] extended this to heterogeneous data center environments where different server generations exhibit distinct thermal transfer characteristics, developing energy-efficient and thermally aware resource management algorithms; their work confirms that uniform thermal thresholds across heterogeneous racks generate persistent hotspots in high-density zones. Nadjahi et al. [9] provided a broad review of data center thermal management and cooling strategies, surveying liquid cooling, airside economization, and software-based thermal control; the review identifies integration with workload scheduling as a persistent open challenge and motivates the need for predictive—rather than reactive—thermal management. Sarkar et al. [34] recently proposed a hierarchical multi-agent framework for carbon-efficient liquid-cooled data center clusters, demonstrating that agent-based coordination of both workload placement and cooling actuation can simultaneously reduce emissions and hotspot duration; however, the method is model-free and lacks formal SLA chance constraints. Goudarzi et al. [28] addressed the intersection of sustainability and thermal management in distributed manufacturing systems, showing that compute node deployment decisions must account for local thermal budgets.

At the chip and embedded system level, Tilli et al. [42] developed a two-layer distributed MPC framework for thermal control of multiprocessor systems-on-chip (MPSoC), demonstrating that MPC's predictive capability enables reliable temperature regulation despite inherent model approximation; this work is directly relevant as proof that MPC is applicable to compute-thermal coupled control, though it operates at the chip level without carbon or SLA dimensions. Wang et al. [11] analyzed thermal implications in energy-agile parallel HPC computing at the node level. A consistent gap across thermal-aware work is the complete absence of carbon-intensity awareness: none of the above frameworks adaptively exploits grid carbon variability in its thermal-workload co-management, nor does any enforce formal probabilistic SLA constraints that bound deadline miss risk under forecast uncertainty.

2.3. Carbon-Aware and Sustainability-Driven Computing

Carbon-aware scheduling exploits the spatiotemporal variability in grid electricity carbon intensity to reduce the carbon footprint of computing without necessarily reducing

throughput. Paul et al. [35] and Lin et al. [21] provide comprehensive reviews of green computing across its past, present, and future dimensions, collectively identifying that most existing approaches address energy or carbon in isolation and that integrated frameworks spanning multiple sustainability objectives remain underexplored. Srivastava et al. [19] examined early green cloud architectures, highlighting the foundational role of workload scheduling in reducing cloud emissions. Zhang et al. [5] introduced RESCUE, among the first schedulers to simultaneously target energy cost and carbon emissions in cloud environments, demonstrating that modest scheduling adjustments can yield meaningful CO_2e reductions when geographic and temporal flexibility is exploited. Le et al. [7] demonstrated that inter-regional workload migration across a network of data centers can jointly reduce electricity cost and carbon footprint, establishing the geo-distribution dimension of carbon-aware scheduling. Khodayarseresht et al. [13] incorporated both energy and carbon cost into the initial VM placement problem for cloud data centers. Oner et al. [14] proposed a combinatorial auction-based VM scheduling model with explicit energy awareness for green cloud computing.

The industrial-scale embodiment of carbon-aware computing is Google's Carbon-Intelligent Compute Management system [36], which delays temporally flexible workloads to low-carbon windows using day-ahead carbon-intensity forecasts and virtual capacity curves (VCCs) with risk-aware sizing that preserves daily throughput capacity. This system demonstrates the production viability of temporal carbon-aware scheduling at hyperscale, but does not jointly optimize power efficiency or model thermal consequences of the shifted utilization profiles. Hall et al. [39] advanced the theoretical underpinnings by applying distributionally robust optimization (DRO) to carbon-aware load shaping for data centers, providing provable performance guarantees under worst-case distributional uncertainty in carbon intensity and compute demand; the framework divides the problem into a day-ahead planning component and a real-time job placement component. Critically, it scopes to temporal load shaping without incorporating thermal constraints or per-job SLA chance constraints. Asadov et al. [38] conducted a thorough literature review of carbon-aware spatio-temporal workload shifting in edge-cloud environments, finding that only 2 of 28 analyzed studies develop fully combined spatial-temporal optimization algorithms and that device production emissions are systematically underweighted; the review explicitly identifies thermal management and formal SLA risk bounds as open research directions. Akour et al. [32] demonstrated that serverless computing models can further reduce environmental impact by eliminating idle compute overheads, while Ananth et al. [29] showed that Green IoT edge computing enables sustainable distributed data processing through localized, low-power pipelines. Pazienza et al. [33] proposed a holistic framework for environmentally sustainable computing that integrates hardware, software, and operational levers, arguing that single-layer optimizations necessarily leave cross-layer savings unrealized.

Gu et al. [47] applied deep reinforcement learning (DRL) to jointly schedule computing resources and green energy for low-carbon edge computing in a model-free approach that adapts continuously to spatially and temporally varying user demand; empirical performance is strong, but the method provides limited formal guarantees on SLA or thermal bounds. Wang et al. [48] proposed ERLFC, a reinforcement-learning framework using the Actor-Critic algorithm for task scheduling in federated cloud environments that exploits carbon emission ratio differences across geographically distributed data centers, achieving competitive energy and emission reductions relative to round-robin and nature-inspired baselines; risk-bounded SLA control is not formalized. Mishra and Singh [37] reviewed Internet of Energy (IoE) frameworks for sustainable smart city energy management, identifying integrated scheduling, cloud computing, and clean energy coordination as key enablers but not addressing chance-constrained SLA control for compute platforms.

2.4. Model Predictive Control for Computing and Energy Systems

MPC's receding-horizon paradigm [15] provides a principled framework for sequential decision-making under forecast uncertainty, with well-understood properties on constraint satisfaction, stability, and sub-optimality from approximated horizons. In the energy and microgrid domain, Huang et al. [40] applied an improved MPC-based optimal scheduling scheme to hydrogen-based microgrids with security constraints, demonstrating reduced daily operating costs and improved computational efficiency. Zheng et al. [41] proposed a multi-scale MPC scheduling strategy for electricity-thermal-hydrogen integrated energy systems, combining a day-ahead optimal dispatch stage with an intra-day rolling correction stage that uses feedback to eliminate tracking errors from renewable energy forecast uncertainty.

In computing systems, Perin et al. [43] developed EASE, an MPC-based energy-aware job scheduler for vehicular edge networks co-powered by renewable energy resources, achieving near-carbon-neutral operation through centralized MPC optimization of local compute resources and renewable energy availability with a distributed consensus step for service migration decisions. This work directly validates MPC's feasibility for edge compute scheduling but does not address thermal dynamics or joint power-carbon-SLA optimization. Perin et al. [44] extended this line to sustainable edge computing with a predictive, online, distributed scheduling algorithm that reduces renewable energy sold back to the grid by up to 50% versus heuristic policies while meeting deadlines; however, carbon-intensity signals and thermal constraints are not modeled. Tilli et al. [42] applied MPC to chip-level thermal control in multiprocessor systems, establishing that a two-layer distributed MPC design achieves reliable temperature regulation. Together, these works confirm MPC's practical viability for energy and

thermal management in computing but leave open the synthesis of joint power-carbon-thermal objectives with formal SLA chance constraints in geo-distributed platforms.

2.5. Robust and Risk-Constrained Optimization

Uncertainty-aware formulations are essential when scheduling decisions must respect constraints under stochastic inputs including demand arrival, runtime estimates, and carbon-intensity forecasts. Nemirovski and Shapiro [16] established convex approximations of chance-constrained programs, providing the theoretical foundation for tractable deterministic reformulations of probabilistic constraints that now forms the core of many robust scheduling designs. Hall et al. [39] applied DRO to carbon-aware data center load shaping, obtaining provable guarantees on scheduling performance under worst-case distributional deviations; however, the scope is temporal load shaping rather than joint power-carbon-thermal multi-objective optimization with per-job SLA chance constraints. Ma et al. [53] demonstrated distributionally robust decarbonizing scheduling for multi-temporal multi-energy microgrids using data-driven ambiguity sets, achieving 10.6% carbon reduction with robust solutions resilient to renewable variability; the application domain is energy grid operation rather than compute scheduling. Dai et al. [54] combined the Column-and-Constraint Generation algorithm with KKT linearization for robust multi-energy virtual power plant scheduling under demand and generation uncertainty, further confirming that explicit uncertainty modeling is tractable for complex energy systems.

These robust optimization works collectively establish that formal uncertainty handling is necessary for reliable sustainable system management. However, chance constraints on per-job SLA deadline violations within a multi-objective power-carbon-thermal scheduling framework for geo-distributed cloud platforms—the precise context addressed by this paper—remain, to the best of our knowledge, unaddressed in the existing literature.

2.6. AI-Driven and Heuristic Resource Management

AI and metaheuristic methods provide a complementary perspective on sustainable cloud and distributed computing. Tuli et al. [49] proposed HUNTER, which combines neural network-based prediction with optimization for holistic sustainable cloud resource management, simultaneously addressing energy, SLA violations, scheduling latency, cost, and temperature; this represents the most comprehensive holistic approach in the cloud scheduling literature but does not provide formal probabilistic SLA guarantees or MPC-based online re-optimization with theoretical stability properties. Wang et al. [48] demonstrated RL-based federated cloud scheduling that effectively exploits geographic carbon emission differences. Kumar et al. [24] surveyed sustainable distributed machine learning paradigms for green edge intelligence, characterizing how data-parallel, model-parallel, and federated approaches differ in their energy profiles and carbon sensitivity; the analysis highlights that training-time scheduling decisions dominate lifetime emissions but that formal risk control during inference is also underexplored. Fard et al. [50] addressed distributed permutation flowshop scheduling under sustainability criteria with real-time control, minimizing makespan with energy consumption constraints. Abbasi et al. [51] demonstrated parallel simulated annealing on HPC infrastructure for virtual power plant scheduling optimization, supporting real-time energy market decision-making. Paul et al. [52] applied quantum particle swarm optimization to grid-connected microgrid management, reducing both operational costs and carbon emissions. While heuristic and learning-based methods demonstrate strong empirical performance on specific benchmarks, they generally lack formal convergence guarantees, explicit chance constraints, and the receding-horizon stability properties that make MPC deployable in production scheduling with certified constraint satisfaction.

2.7. Gap Analysis and Positioning

Table 1 provides a structured comparison of representative related works across six key dimensions: energy awareness, carbon awareness, thermal awareness, joint multi-objective optimization, risk or chance constraints, and MPC-based online receding-horizon control. Three high-level observations emerge from this review. First, the majority of existing work addresses only one or two of these dimensions simultaneously, with power-only and carbon-only approaches dominating the literature—a finding reinforced by the comprehensive reviews of Zakarya et al. [18] and Paul et al. [35] who survey enabling models and techniques across the green computing landscape. Second, thermal modeling and formal SLA chance constraints are the two dimensions most consistently absent even in multi-objective frameworks. Third, while MPC has been successfully applied to energy scheduling in edge computing and to chip-level thermal control, its use in joint power-carbon-thermal scheduling at the geo-distributed cloud level is absent. Peltonen et al. [26] argue from a systems-design perspective that the computing community must treat climate sustainability as a first-class requirement rather than a post-hoc optimization target, precisely the design philosophy that motivates SustainSched-MPC.

Specifically, no existing framework simultaneously provides: (i) joint optimization of power, carbon emissions, and thermal stress; (ii) online MPC-based receding-horizon control; (iii) explicit chance constraints on per-job SLA deadline violations; (iv) multi-region geo-distributed deployment; and (v) a systematic multi-day evaluation covering all these dimensions against multiple named baselines. SustainSched-MPC is designed to fill this five-way intersection, and Section 9 quantifies the performance impact of each missing dimension through ablation and baseline comparisons.

Table 1
Comparison of representative related works across six sustainability and control dimensions. ✓ = fully addressed; **P** = partially addressed; – = not addressed. SustainSched-MPC (last row, shaded) is, to our knowledge, the only method in this comparison satisfying all six dimensions simultaneously.

Work	Energy-aware	Carbon-aware	Thermal-aware	Multi-objective	Chance/Risk Constr.	MPC Receding Horizon
Beloglazov et al. [1]	✓	–	–	–	–	–
Kliazovich et al. [2]	✓	–	–	–	–	–
Gai et al. [30]	✓	–	P	–	–	–
Dorronsoro et al. [6]	✓	–	–	✓	–	–
Li [10]	✓	–	–	–	–	–
Borghesi et al. [8]	✓	–	–	–	–	–
Kocot et al. [45]	✓	–	–	–	–	–
Banerjee et al. [3]	P	–	✓	–	–	–
Sun et al. [4]	✓	–	✓	P	–	–
Tilli et al. [42]	–	–	✓	–	–	✓
Sarkar et al. [34]	–	✓	✓	P	–	–
Le et al. [7]	✓	✓	–	P	–	–
Radovanovic et al. [36]	–	✓	–	–	P	–
Hall et al. [39]	–	✓	–	–	✓	–
Gu et al. [47]	✓	✓	–	P	–	–
Perin et al. [44]	✓	P	–	–	–	✓
Perin et al. [43]	✓	P	–	–	–	✓
Tuli et al. [49]	✓	–	✓	✓	–	–
Cao et al. [46]	✓	–	–	P	–	–
Sharma et al. [25]	✓	✓	–	P	–	–
SustainSched-MPC (ours)	✓	✓	✓	✓	✓	✓

3. System Model and Assumptions

This section formally defines the infrastructure, workload, network, and uncertainty models that underpin SustainSched-MPC. The model is intentionally grounded in quantities measurable from standard data center telemetry—IPMI power readings, DCIM temperature sensors, carbon-intensity APIs, and job-scheduler queue logs—so that all parameters can be calibrated from operational data without bespoke instrumentation.

3.1. Infrastructure Model

Spatial decomposition. The infrastructure consists of $R = |\mathcal{R}|$ geographic *regions*, indexed $r \in \mathcal{R}$. Each region r contains a set of physical *hosts* $\mathcal{H}_r$, and the global host set is $\mathcal{H} = \bigcup_r \mathcal{H}_r$ with $|\mathcal{H}| = H$. Regions correspond to distinct physical data center sites or cloud availability zones whose interconnectivity is characterized by finite-bandwidth, latency-bounded links. The combined host count across our experimental platform is $H = 24$ across $R = 3$ regions, reflecting a representative medium-scale geo-distributed cluster.

Host resource model. Each host $h \in \mathcal{H}$ is characterized by a fixed resource capacity vector

$$\mathbf{K}_h = \left(K_h^{\text{cpu}},\ K_h^{\text{mem}},\ K_h^{\text{io}}\right) \in \mathbb{R}_+^3, \tag{1}$$

where the three components represent peak CPU thread-slots (normalized to 1 GHz-equivalent cores), memory bandwidth (GB/s), and I/O bandwidth (Gbps). At each control interval $t \in \mathcal{T} = \{1, 2, \ldots, T\}$ of duration $\Delta = 5\,\text{min}$, the *effective* capacity is modulated by the current DVFS frequency state $f_{h,t} \in \mathcal{F}_h$, where $\mathcal{F}_h = \{f_h^{(1)}, \ldots, f_h^{(L)}\}$ is the finite set of supported P-states (typically 4–6 discrete levels between 1.2 and 3.6 GHz):

$$\mathbf{K}_{h,t} = \mathbf{K}_h \cdot (f_{h,t}/f_h^{\max}), \tag{2}$$

where $f_h^{\max}$ is the host's rated maximum frequency. This captures the linear throughput reduction associated with downclocking; the quadratic power reduction is accounted for in the power model (Section 4). A host may also be placed in a low-power sleep state $s_{h,t} \in \{0, 1\}$, where $s_{h,t} = 0$ means the host is in standby (drawing only idle power, no jobs can run) and $s_{h,t} = 1$ means it is active.

Power states. The feasible power envelope of each host is described by the triple $(P_h^{\text{idle}}, \alpha_h, \beta_h)$:

- P_h^{idle} (W): baseline power drawn when the host is active but fully idle (no assigned jobs); drawn from IPMI out-of-band readings at zero utilization.
- α_h (W/%): marginal power increase per percentage point of CPU utilization, capturing the compute-intensive portion of dynamic power.
- β_h (W/GHz): power sensitivity to DVFS state, reflecting the super-linear voltage-scaling effect on dynamic power.

These parameters are calibrated per host from a 7-day telemetry baseline prior to the experiment window. Host

power values in our setup range from 86 W (idle, underclocked) to 215 W (peak utilization, max frequency), consistent with mid-generation enterprise server specifications.

Cooling and PUE. Cooling energy is accounted via a per-region Power Usage Effectiveness (PUE) factor $\rho_r \geq 1$, so that the total facility energy associated with compute power $P_{h,t}$ is $\rho_r \cdot P_{h,t}$. The PUE values are assumed constant within each control interval but may vary across regions (e.g., $\rho_r = 1.3$ for an air-cooled US facility, $\rho_r = 1.1$ for a liquid-cooled European site). This is a standard accounting boundary consistent with prior sustainable computing literature [3, 9].

3.2. Workload Model

Job model. The system manages a stream of compute *jobs* $j \in \mathcal{J}$. Each job is described by the tuple

$$j = \big(a_j,\ d_j,\ \tau_j,\ \mathbf{u}_j,\ s_j,\ \delta_j\big), \tag{3}$$

where:

- $a_j \in \mathcal{T}$: *release time* (earliest slot the job can start), derived from the job's submit timestamp.
- $d_j \in \mathcal{T}$: *deadline* (latest slot by which the job must *complete*), derived from SLA policy for the job's class.
- $\tau_j > 0$: *estimated runtime* (in slots), obtained from a calibrated runtime predictor; since τ_j is an estimate, it is treated as a random variable with mean $\bar{\tau}_j$ and coefficient of variation $\sigma_j/\bar{\tau}_j$ (see Section 3.4).
- $\mathbf{u}_j = (u_j^{\text{cpu}}, u_j^{\text{mem}}, u_j^{\text{io}}) \in \mathbb{R}_+^3$: *resource demand vector* (in the same units as $\mathbf{K}_h$), drawn from profiling data for the job's application type.
- $s_j \in \mathcal{S}$: *SLA class*, partitioning jobs into a finite set $\mathcal{S}$ of service tiers with distinct deadline risk tolerances ϵ_s (defined in Section 5).
- $\delta_j \in [0, d_j - a_j]$: *maximum deferral* (in slots), the operator-configured window within which the scheduler may delay a delay-tolerant job beyond its natural earliest start; $\delta_j = 0$ for latency-critical jobs, which must start as soon as resources allow.

SLA classes and workload mix. In our experimental setup, two SLA classes are defined. Class $s = 1$ ("Batch/analytics", 70% of jobs by count) has generous deadlines of 4–8 h from release and allows δ_j up to 60 min, enabling the scheduler to shift these jobs to low-carbon, low-temperature windows. Class $s = 2$ ("Latency-critical services", 30%) has strict deadlines of 15–30 min from release and $\delta_j = 0$. The risk tolerance parameters are set to $\epsilon_1 = 2.0\%$ and $\epsilon_2 = 0.5\%$, respectively.

Job arrival process. Jobs arrive continuously and are observed by the scheduler at each 5-minute receding-horizon step. The arrival rate follows a non-stationary Poisson process with a time-varying intensity λ_t that tracks the daily demand profile shown in Fig. 1, peaking between 09:00–14:00 local time. On average, 18,240 jobs/day arrive across all regions, with a peak-to-trough ratio of approximately 5:1. The scheduler does not have access to future arrivals; it uses short-horizon demand forecasts described in Section 3.4.

Non-preemptive execution assumption. Jobs are assumed non-preemptive: once a job starts on host h at interval t, it occupies its allocated resource fraction on that host for τ_j consecutive intervals and cannot be interrupted or migrated mid-execution. This assumption is standard in batch and HPC scheduling [45] and is consistent with the execution semantics of containerized workloads where checkpoint-restart overhead is prohibitive for short jobs.

3.3. Inter-Region Network and Migration Model

Migration bandwidth. Live workload migration (relocating a queued—but not yet started—job from region r to region r') incurs a data-transfer overhead modeled as a reduction in the effective scheduling window. Specifically, if region r' is chosen for job j instead of its region of origin r_j^*, the effective release time is advanced by a migration latency $\ell_{r,r'} \geq 0$ (in slots):

$$a_j^{\text{eff}}(r') = a_j + \ell_{r_j^*,r'}. \tag{4}$$

Migration latencies $\ell_{r,r'}$ are pre-computed from inter-region bandwidth measurements and the job's input data size. In our three-region setup, $\ell_{r,r'} \in \{0, 1, 2\}$ slots (0–10 min), reflecting co-located, same-continent, and cross-continent pairings, respectively.

Migration quota. To prevent oscillatory migration behavior—a known instability in unconstrained migration optimizers—the scheduler is subject to a per-interval migration quota $M_{\max}$: the total number of jobs simultaneously migrating across all region pairs at interval t is bounded:

$$\sum_{r \neq r'} m_{r,r',t} \leq M_{\max}, \tag{5}$$

where $m_{r,r',t}$ counts inter-region migrations initiated at t. Explicitly, $m_{r,r',t}$ is linked to job assignment decisions by:

$$m_{r,r',t} = \sum_{j \in \mathcal{J}\,:\, r_j^* = r} \sum_{h \in \mathcal{H}_{r'}} x_{j,r',h,t}, \quad r \neq r', \tag{6}$$

where r_j^* is job j's region of origin (i.e., the region where it was released or last assigned). The migration count $m_{r,r',t}$ thus exactly counts cross-region assignments at each interval, and the quota constraint $\sum_{r \neq r'} m_{r,r',t} \leq M_{\max}$ bounds total migration throughput. In our experiments $M_{\max} = 5$ per interval, limiting total migration to below 2% of scheduling throughput.

3.4. Uncertainty and Forecast Model

Carbon-intensity forecasts. The carbon intensity of grid electricity in region r at interval t, $I_{r,t}$ (gCO_2e/kWh), is

obtained from a short-term forecast $\hat{I}_{r,t}$ with additive error:

$$I_{r,t} = \hat{I}_{r,t} + \xi^C_{r,t}, \quad \xi^C_{r,t} \sim \mathcal{N}(0, \sigma^2_{C,r}), \tag{7}$$

where $\sigma_{C,r}$ is the region-specific one-step-ahead forecast standard deviation, estimated from historical forecast residuals. In our setup, mean absolute percentage error (MAPE) of the one-hour carbon forecast ranges from 4% to 9% across the three regions, consistent with publicly reported accuracy for marginal emission factor forecasts [36].

Runtime estimation uncertainty. The estimated runtime $\bar{\tau}_j$ for job j is produced by an application-profile-based predictor. The true runtime is modeled as:

$$\tau_j = \bar{\tau}_j(1 + \xi^\tau_j), \quad \xi^\tau_j \sim \mathcal{N}(0, \sigma^2_\tau), \tag{8}$$

where σ_τ is calibrated per application class from historical prediction residuals (typically 8–18% relative error for batch analytics, 4–6% for short latency-critical tasks). Runtime uncertainty directly propagates into SLA deadline risk: a job that takes longer than predicted may miss its deadline even if it started on time, which is why formal chance constraints are necessary (Section 5).

Ambient temperature forecasts. Node cooling effectiveness depends on the ambient air or coolant inlet temperature $T^{\text{amb}}_{h,t}$, which varies with external weather, HVAC state, and neighboring rack load. A short-term forecast $\hat{T}^{\text{amb}}_{h,t}$ with error $\xi^T_{h,t} \sim \mathcal{N}(0, \sigma^2_T)$ (typically ±1–2 °C over a 60-min horizon) is used in the thermal model.

Demand arrival forecasts. The scheduler uses a short-horizon demand forecast $\hat{\lambda}_{t+k}$ for $k = 1, \ldots, H_p$ (where $H_p = 12$ is the MPC prediction horizon) to anticipate incoming job load during the optimization window. The demand forecast is produced by a lightweight autoregressive model fitted to the trailing 7 days of queue arrival history, with MAPE below 11% at a 60-min horizon.

3.5. Key Assumptions and Scope Boundaries

The model operates under the following explicit assumptions, which define the scope and reproducibility boundary of SustainSched-MPC:

1. **Homogeneous within-region network:** Intra-region host-to-host communication latency is negligible relative to the 5-minute control interval, so all hosts within a region share the same effective locality for job placement.
2. **Independent host failures:** Host failures are not explicitly modeled; the system relies on the surrounding infrastructure (hypervisor watchdog, health-check APIs) to remove failed hosts from the schedulable pool between intervals. Fault-tolerant scheduling extensions are deferred to future work.
3. **Stationarity of power model parameters:** The calibrated power parameters ($\alpha_h, \beta_h, P^{\text{idle}}_h$) are assumed constant over the 30-day evaluation window. Long-term hardware aging effects are negligible at this timescale.
4. **First-order RC thermal dynamics:** The thermal model (Section 4.2) assumes linear heat conduction; radiative coupling between adjacent racks is subsumed into the empirically calibrated thermal resistance parameter κ_h.
5. **Operational (Scope 2) carbon accounting only:** Carbon emissions are computed from electricity purchase (grid-mix emission factors), consistent with Scope 2 GHG Protocol accounting. Embodied (Scope 3) manufacturing emissions are excluded.
6. **No virtual machine live migration mid-execution:** As noted above, only queued jobs (not yet started) may be migrated across regions. In-flight workload relocation is outside the model scope.

Table 2
Principal notation for the system model.

Symbol	Definition
$\mathcal{R}$, r	Set of regions; region index
$\mathcal{H}$, h	Set of hosts; host index
$\mathcal{T}$, t	Set of control intervals; interval index
Δ	Control interval duration (5 min)
$\mathbf{K}_h$	Static capacity vector of host h
$\mathbf{K}_{h,t}$	DVFS-modulated effective capacity
$f_{h,t}$	DVFS frequency state of host h at t
$s_{h,t}$	Active/standby binary indicator
P^{idle}_h, α_h, β_h	Power model parameters
ρ_r	Region PUE cooling factor
j, $\mathcal{J}$	Job; global job set
a_j, d_j	Job release time and deadline (slots)
τ_j, $\bar{\tau}_j$	True and estimated job runtime
$\mathbf{u}_j$	Job resource demand vector
s_j	SLA class of job j
δ_j	Maximum scheduling deferral (slots)
$\mathcal{S}$, ϵ_s	SLA class set; per-class risk budget
$\hat{I}_{r,t}$	Carbon-intensity forecast (gCO_2e/kWh)
$T_{h,t}$	Node temperature at interval t (°C)
$T^{\text{amb}}_{h,t}$	Ambient temperature forecast
η_h, κ_h	Thermal gain and cooling coefficients
$\ell_{r,r'}$	Inter-region migration latency (slots)
$M_{\max}$	Per-interval migration quota

Table 2 summarizes all principal symbols introduced in this section for reference throughout the paper.

4. Power, Thermal, and Carbon Modeling

This section develops the three quantitative sub-models that together constitute the physics layer of SustainSched-MPC: (i) a DVFS-parameterized power-performance model that maps workload placement and frequency decisions to real-time power draw; (ii) a first-order RC thermal model

that propagates power draw to node temperature dynamics; and (iii) a carbon emission model that converts facility energy consumption to operational CO_2e using time-varying regional grid intensity forecasts. All three models are designed to be (a) calibratable from standard infrastructure telemetry without bespoke sensors, (b) evaluable in closed form at each 5-minute MPC step, and (c) compositional so that they can be embedded jointly into the chance-constrained optimizer described in Section 5.

4.1. Power–Performance Model

Physical motivation. Server power consumption has two physically distinct components: *static* (leakage) power that is always drawn whenever the host is active, and *dynamic* power that scales with computational activity and operating voltage/frequency. Empirical measurements on modern x86 servers consistently show that the total power draw is well-approximated by an affine function of CPU utilization over the range from idle to full load [1, 45], with an additional frequency-dependent term that captures the cubic voltage-frequency relationship of CMOS circuits. The model below captures both effects while remaining analytically tractable.

Base power model. For host $h \in \mathcal{H}$ in active state ($s_{h,t} = 1$) at interval t, the instantaneous power draw is:

$$P_{h,t} = P_h^{\text{idle}} + \alpha_h\, u_{h,t} + \beta_h\, \phi(f_{h,t}), \tag{9}$$

where:

- P_h^{idle} (W) is the *static idle power*: the baseline draw when the host is fully active but executing no scheduled jobs. It includes memory refresh, chipset leakage, and fan overhead, and is measured from IPMI readings at zero CPU utilization.
- $u_{h,t} \in [0, 1]$ is the aggregate *CPU utilization fraction* at interval t, defined as the ratio of allocated CPU demand to total host capacity: $u_{h,t} = \sum_j x_{j,r,h,t}\, u_j^{\text{cpu}} / K_h^{\text{cpu}}$.
- α_h (W/unit utilization) is the *utilization sensitivity*: the slope of the linear power–utilization curve between idle and peak, calibrated from a linear regression on telemetry samples collected across the full utilization range.
- $f_{h,t} \in \mathcal{F}_h$ is the selected *DVFS P-state* (frequency in GHz).
- $\phi(f_{h,t}) = (f_{h,t}/f_h^{\max})^3$ is the *normalized frequency cube*, reflecting the well-known cubic scaling of CMOS dynamic power with operating frequency (since $P_{\text{dyn}} \propto CV^2 f$ and $V \propto f$ in DVFS operation, giving $P_{\text{dyn}} \propto f^3$) [10].
- β_h (W) is the *frequency power coefficient*: the additional dynamic power attributable to running at maximum frequency ($f_{h,t} = f_h^{\max}$) beyond the utilization-linear term.

When a host is in standby ($s_{h,t} = 0$), no jobs are assigned and the power draw is a fixed low standby value $P_h^{\text{stby}} \leq P_h^{\text{idle}}$, which accounts for power-supply idling and baseboard management controller (BMC) operation:

$$P_{h,t} = s_{h,t}\left[P_h^{\text{idle}} + \alpha_h\, u_{h,t} + \beta_h\, \phi(f_{h,t})\right] + (1 - s_{h,t})\, P_h^{\text{stby}}. \tag{10}$$

Energy per interval. The energy consumed by host h over control interval t (of duration $\Delta = 300\,\text{s}$) is:

$$E_{h,t} = \frac{P_{h,t} \cdot \Delta}{3{,}600{,}000} \quad (\text{kWh}), \tag{11}$$

The total cluster energy over the optimization horizon $\mathcal{T}$ is $E = \sum_{h,t} E_{h,t}$, which forms the energy term of the multi-objective cost function.

Parameter calibration. For each host h, the triple $(P_h^{\text{idle}}, \alpha_h, \beta_h)$ is calibrated from a 7-day pre-experiment baseline by running controlled microbenchmarks at three utilization levels (0%, 50%, 100%) and three P-states (minimum, mid, maximum). Ordinary least-squares regression on the resulting 9 operating points yields $R^2 > 0.97$ in all cases, confirming the model adequacy. The resulting parameter ranges for our 24-node cluster are: $P_h^{\text{idle}} \in [86, 112]\,\text{W}$, $\alpha_h \in [65, 98]\,\text{W}$, $\beta_h \in [12, 22]\,\text{W}$.

4.2. Thermal Model

Physical motivation. Server junction temperatures evolve on timescales of seconds to minutes, making them observable at the 5-minute control interval but fast enough to be affected by within-interval workload bursts. The thermal dynamics of a computing node can be well-approximated by a lumped first-order RC (resistor-capacitor) model [42, 3], in which the node is treated as a thermal mass (capacitance) C_h exchanging heat with the ambient environment through a thermal resistance R_h. This model is standard in chip-level thermal control and has been validated against physical measurements at the server level in controlled data center environments.

Discrete-time RC model. Discretizing the continuous-time first-order ODE $C_h \dot{T}_h = P_h(t) - (T_h - T_h^{\text{amb}})/R_h$ at the control interval Δ using forward Euler integration gives the update equation:

$$T_{h,t+1} = T_{h,t} + \underbrace{\eta_h\, P_{h,t}}_{\text{heat input}} - \underbrace{\kappa_h\left(T_{h,t} - T_{h,t}^{\text{amb}}\right)}_{\text{cooling dissipation}}, \tag{12}$$

where:

- $T_{h,t}$ (°C) is the *observed node temperature* at the start of interval t, read from DCIM inlet/outlet or CPU package temperature sensors.
- $\eta_h = \Delta / C_h$ (K/W per interval) is the *thermal gain coefficient*: the temperature rise per watt of power input per interval, inversely proportional to the effective thermal capacitance (mass × specific heat) of the server chassis.

- $\kappa_h = \Delta/(R_h C_h) \in (0, 1)$ (per interval) is the *cooling coefficient*: the fraction of the current temperature excess above ambient that is dissipated per interval. It is bounded strictly below 1 for numerical stability of the forward Euler scheme.
- $T^{\text{amb}}_{h,t}$ (°C) is the *ambient (inlet) temperature* at interval t, obtained from the forecast model defined in Section 3.4.

Steady-state and hotspot threshold. Under constant power P_h and constant ambient T^{amb}_h, the steady-state temperature reached by Eq. (12) is:

$$T^{\text{ss}}_h = T^{\text{amb}}_h + \frac{\eta_h}{\kappa_h} P_h = T^{\text{amb}}_h + R_h P_h, \tag{13}$$

which recovers the standard thermal resistance formula. SustainSched-MPC enforces a *predicted thermal cap* $T^{\max} = 82\,°\text{C}$: if any predicted $T_{h,t+k}$ (for $k = 1, \ldots, H_p$) within the horizon exceeds $T^{\max}$, the optimizer is constrained to reduce power or migrate workload on that host (Section 5). Operationally, 82 °C is chosen as a 3 °C margin below the typical hardware throttling onset of 85 °C.

Thermal stress metric. Short-duration exceedances above 80 °C (hotspot events) are penalized in the objective as thermal stress Θ:

$$\Theta = \sum_{h,t} \max\big(0,\ T_{h,t} - T^{\text{hot}}\big) \cdot \Delta, \tag{14}$$

where $T^{\text{hot}} = 80\,°\text{C}$ is the hotspot threshold and the product with Δ converts the temperature excess to degree-minutes of hotspot exposure. This metric penalizes both the magnitude and duration of thermal exceedances and is a continuous approximation that permits gradient-based optimization.

Parameter calibration. Thermal parameters (η_h, κ_h) are calibrated per host by fitting the discrete RC model to 48-hour temperature traces at varied utilization levels. The calibration solves a linear least-squares problem: given observed pairs $(P_{h,t}, T_{h,t}, T_{h,t+1}, T^{\text{amb}}_{h,t})$, regress $T_{h,t+1} - T_{h,t}$ onto $P_{h,t}$ and $(T_{h,t} - T^{\text{amb}}_{h,t})$. Calibrated values in our cluster span $\eta_h \in [0.0012, 0.0031]$ K/W and $\kappa_h \in [0.041, 0.076]$ per interval, yielding thermal time constants $\tau^T_h = 1/\kappa_h \in [13, 24]$ intervals (65–120 min), consistent with published server-level thermal inertia measurements.

4.3. Carbon Emission Model

Physical motivation. The carbon footprint of computing operations depends on both the quantity of electricity consumed and the marginal emission intensity of the grid supplying that electricity. Grid carbon intensity varies substantially across geographic regions—by a factor of 5–10× between renewable-heavy and fossil-heavy grids—and temporally within a region due to renewable generation intermittency and demand response [36, 7]. Exploiting this variability through spatial workload migration and temporal deferral is the central mechanism of carbon-aware scheduling.

Marginal emission intensity. Let $I_{r,t}$ (gCO_2e/kWh) denote the *marginal emission intensity* of the electricity grid supplying region r at interval t. SustainSched-MPC uses a short-term forecast $\hat{I}_{r,t}$ obtained from a grid carbon API (e.g., Electricity Maps, WattTime) with one-hour-ahead lookahead. The forecast error is modeled as additive Gaussian noise as defined in Section 3.4. The choice of marginal (rather than average) intensity is consistent with best practice in operational carbon accounting for flexible loads [36, 39], since marginal intensity reflects the actual generation that would change as a result of an incremental load shift.

Operational carbon per host per interval. The CO_2e attributed to host h in region r at interval t is:

$$C_{h,r,t} = \frac{\rho_r \cdot E_{h,t} \cdot \hat{I}_{r,t}}{1000} \quad (\text{kgCO}_2\text{e}), \tag{15}$$

where $E_{h,t}$ is in kWh (Eq. (11)), $\hat{I}_{r,t}$ is in gCO_2e/kWh, and $\rho_r \geq 1$ is the regional PUE. The product $E_{h,t} \cdot \hat{I}_{r,t}$ yields gCO_2e; dividing by 1000 converts to $kgCO_2e$. Summing over the full cluster and horizon gives the total operational carbon:

$$C = \sum_{r \in \mathcal{R}} \sum_{h \in \mathcal{H}_r} \sum_{t \in \mathcal{T}} C_{h,r,t}. \tag{16}$$

Spatiotemporal carbon intensity profile. Fig. 1 illustrates the representative normalized daily profiles for workload demand and carbon intensity used in our experiments. The anti-correlation between peak demand (09:00–14:00) and minimum carbon intensity (10:00–16:00, when solar generation is highest) is the primary carbon savings opportunity exploited by SustainSched-MPC: batch workloads that can be deferred by up to 60 minutes are shifted into the midday low-carbon window, and inter-region migration directs workload to the lowest-intensity region available within the migration quota.

Forecast uncertainty handling. Since $\hat{I}_{r,t}$ is a prediction, the realized carbon may differ from the predicted value. In the MPC framework, the carbon cost in the objective uses the point forecast $\hat{I}_{r,t}$, while the chance constraints (Section 5) account for the forecast error $\xi^C_{r,t}$ through reserve margins calibrated from historical residuals. This separates the carbon minimization objective (deterministic, using best estimates) from the SLA reliability constraints (stochastic, requiring risk-adjusted margins).

4.4. Cooling Energy and PUE Accounting

PUE decomposition. The Power Usage Effectiveness factor ρ_r used in Eq. (15) is defined as:

$$\rho_r = \frac{P^{\text{total}}_r}{P^{\text{IT}}_r}, \tag{17}$$

where P^{IT}_r is the total IT equipment power draw (servers, storage, network) and P^{total}_r includes cooling, uninterruptible power supply (UPS) losses, and building services power.

From the scheduling perspective, ρ_r acts as a multiplicative amplifier: a 1 kWh reduction in server energy yields a ρ_r kWh reduction in facility energy, making high-PUE regions disproportionately beneficial targets for workload reduction.

Thermal-PUE coupling. A subtlety not captured by a fixed ρ_r is that cooling power—and therefore PUE—is itself a function of server inlet temperatures. At higher average rack temperatures, cooling systems (CRACs, chillers) must operate at higher capacity, increasing PUE. A simple linear correction can be applied:

$$\rho_r(T_t) = \rho_r^0 + \gamma_r \cdot \max\big(0,\ \bar{T}_{r,t} - T^{\text{ref}}\big), \tag{18}$$

where $\bar{T}_{r,t}$ is the mean rack temperature in region r at interval t, T^{ref} is the nominal operating temperature (22 °C), and γ_r (increase in PUE per degree Celsius above reference) is a site-specific parameter. In our experimental setup, we use fixed $\rho_r \in \{1.1, 1.2, 1.3\}$ for the three regions (liquid-cooled, mixed, air-cooled), consistent with the operating temperature ranges observed during the 30-day evaluation. The dynamic correction would amplify the thermal penalty of hotspot events and is left as an extension.

4.5. Consistency and Model Coupling

The three sub-models are coupled through the shared decision variables $(x_{j,r,h,t}, f_{h,t}, s_{h,t})$:

1. **Placement → power:** $x_{j,r,h,t}$ determines $u_{h,t}$ (Eq. (9)), which drives $P_{h,t}$.
2. **Frequency → power and throughput:** $f_{h,t}$ appears directly in the power model (Eq. (9)) and reduces effective capacity $\mathbf{K}_{h,t}$, coupling energy efficiency to feasibility.
3. **Power → temperature:** $P_{h,t}$ drives the RC thermal update (Eq. (12)), making temperature a dependent state variable.
4. **Power and region → carbon:** $P_{h,t}$ determines $E_{h,t}$, which combined with $\hat{I}_{r,t}$ and ρ_r gives $C_{h,r,t}$ (Eq. (15)).

This coupling means that any scheduling decision simultaneously affects all four quantities (energy, temperature, carbon, and throughput), which is why a joint optimization across all four is necessary. Single-objective optimizers operating on only one of these quantities generate compensating degradations in the others—a claim quantitatively validated by the ablation analysis in Section 9.

5. Problem Formulation

SustainSched-MPC addresses a multi-objective, risk-constrained job-scheduling problem over a finite MPC horizon. This section presents the complete formal problem description, including the decision variables, objective function, hard feasibility constraints, and stochastic SLA chance constraints. All notation follows Table 2.

5.1. Decision Variables

The optimizer controls three classes of variables at each receding-horizon step:

Job placement variables. The primary scheduling decision is encoded in the binary tensor:

$$x_{j,r,h,t} \in \{0,1\}, \quad j \in \mathcal{J},\ r \in \mathcal{R},\ h \in \mathcal{H}_r,\ t \in \mathcal{T}, \tag{19}$$

where $x_{j,r,h,t} = 1$ if and only if job j is *started* on host h in region r at control interval t. Because jobs are non-preemptive, a single $x_{j,r,h,t} = 1$ entry fully determines the job's resource occupancy for slots t through $t + \tau_j - 1$ on the assigned host.

DVFS frequency variables. Each active host is assigned a P-state at every interval:

$$f_{h,t} \in \mathcal{F}_h = \{f_h^{(1)}, \dots, f_h^{(L)}\}, \quad h \in \mathcal{H},\ t \in \mathcal{T}. \tag{20}$$

In the MPC optimization, $f_{h,t}$ is relaxed to the convex hull $[f_h^{\min}, f_h^{\max}]$ and then projected to the nearest feasible P-state at the commit step, enabling efficient linear programming while preserving hardware feasibility.

Host power-state variables.

$$s_{h,t} \in \{0,1\}, \quad h \in \mathcal{H},\ t \in \mathcal{T}, \tag{21}$$

where $s_{h,t} = 1$ indicates host h is active at interval t (eligible for job assignment) and $s_{h,t} = 0$ places it in standby. Standby-to-active wake-up incurs a 1-slot unavailability penalty. Let binary variable $w_{h,t} = 1$ denote that host h wakes from standby at interval t (i.e., $s_{h,t} = 1$ and $s_{h,t-1} = 0$). The wake event is encoded by:

$$w_{h,t} \geq s_{h,t} - s_{h,t-1}, \quad w_{h,t} \in \{0,1\}, \tag{22}$$

and job assignment is forbidden on the first active slot after wake-up:

$$\sum_{j,r} x_{j,r,h,t} \leq (1 - w_{h,t}) \cdot |\mathcal{J}|, \tag{23}$$

ensuring no job is dispatched to h in the same interval it wakes. This adds $H \cdot H_p$ binary variables and $2H \cdot H_p$ linear constraints.

Auxiliary migration variables.

$$m_{r,r',t} \in \mathbb{Z}_{\geq 0}, \quad r \neq r',\ t \in \mathcal{T}, \tag{24}$$

counting cross-region migrations initiated at interval t, subject to the quota constraint of Section 3.3.

5.2. Multi-Objective Cost Function

The scheduler minimizes a weighted combination of four sustainability and service objectives over the H_p-slot prediction horizon $[t_0, t_0 + H_p)$:

$$\min_{x,f,s,m} \quad \mathcal{L} = \lambda_E E + \lambda_C C + \lambda_T \Theta + \lambda_D D, \tag{25}$$

where each term is defined as follows.

Energy term E (kWh). Total facility energy consumption over the horizon, including PUE-amplified server power:

$$E = \sum_{t=t_0}^{t_0+H_p-1} \sum_{r\in\mathcal{R}} \sum_{h\in\mathcal{H}_r} \rho_r \, P_{h,t}(x, f, s) \cdot \frac{\Delta}{3{,}600{,}000}, \quad (26)$$

with $P_{h,t}$ given by Eq. (10).

Carbon term C ($kgCO_2e$). Total operational carbon emissions:

$$C = \sum_t \sum_r \sum_{h\in\mathcal{H}_r} \frac{\rho_r \, P_{h,t} \cdot \frac{\Delta}{3{,}600{,}000} \cdot \hat{I}_{r,t}}{1000}. \quad (27)$$

Thermal stress term Θ (degree-minutes). Cumulative hotspot exposure above the 80 °C threshold:

$$\Theta = \sum_t \sum_h \max\big(0,\ T_{h,t} - T^{\text{hot}}\big) \cdot \Delta, \quad (28)$$

where $T_{h,t}$ evolves according to the RC model (12).

Delay penalty term D (slot-seconds). For batch (delay-tolerant) jobs, a weighted scheduling delay beyond the natural earliest start:

$$D = \sum_{j\in\mathcal{J}_{s=1}} \sum_t \sum_{r,h} x_{j,r,h,t} \cdot (t - a_j) \cdot \Delta. \quad (29)$$

For latency-critical jobs ($s = 2$, $\delta_j = 0$), no deferral is permitted (captured in constraint (32)).

Weight selection. The weights $(\lambda_E, \lambda_C, \lambda_T, \lambda_D)$ are operator-configured hyperparameters that encode the relative importance of each objective. In our experiments, the baseline configuration is $(\lambda_E, \lambda_C, \lambda_T, \lambda_D) = (0.15, 0.55, 0.20, 0.10)$, placing the highest priority on carbon, followed by thermal safety, energy, and delay. A sensitivity analysis over weight combinations is reported in Section 9.

5.3. Hard Feasibility Constraints

Single-assignment constraint. Each job is started exactly once within the feasible scheduling window $[a_j, d_j - \tau_j]$:

$$\sum_{r\in\mathcal{R}} \sum_{h\in\mathcal{H}_r} \sum_{t=a_j}^{d_j-\tau_j} x_{j,r,h,t} = 1, \quad \forall j \in \mathcal{J}. \quad (30)$$

Capacity constraint. The aggregate resource demand of all concurrent jobs on host h at interval t must not exceed the DVFS-modulated effective capacity:

$$\sum_{j\in\mathcal{J}} \sum_{\tau=0}^{\tau_j-1} x_{j,r,h,t-\tau} \cdot \mathbf{u}_j \ \le\ \mathbf{K}_{h,t}(f_{h,t}), \quad \forall r, h, t, \quad (31)$$

where the left-hand sum aggregates demand from all jobs whose execution overlaps interval t (started at any prior slot $t - \tau_j < t$).

Deadline feasibility constraint. For latency-critical (non-deferrable) jobs, the job must both start and complete before its deadline:

$$x_{j,r,h,t} = 0 \quad \text{if } t < a_j \ \text{ or } \ t + \tau_j > d_j, \quad \forall j \in \mathcal{J}_{s=2}. \quad (32)$$

For batch jobs, the flexibility window is $[a_j, a_j + \delta_j]$: $x_{j,r,h,t} = 0$ for $t > a_j + \delta_j$.

Host utilization feasibility. A job can only be assigned to an active host:

$$\sum_{j,\tau} x_{j,r,h,t-\tau} \le s_{h,t} \cdot |\mathcal{J}|, \quad \forall h, t. \quad (33)$$

Thermal hard cap. To prevent hardware throttling, predicted node temperature must not exceed the thermal cap $T^{\max}$:

$$T_{h,t} \le T^{\max} = 82\,°\text{C}, \quad \forall h \in \mathcal{H},\ t \in \mathcal{T}. \quad (34)$$

Migration quota.

$$\sum_{r\neq r'} m_{r,r',t} \le M_{\max} = 5, \quad \forall t \in \mathcal{T}. \quad (35)$$

5.4. Stochastic SLA Chance Constraints

Chance constraint formulation. Let T_j^{comp} denote the actual completion time of job j, which depends on the scheduled start time and the stochastic runtime $\tau_j = \bar{\tau}_j(1+\xi_j^\tau)$. The SLA deadline guarantee for class s is expressed as a chance constraint:

$$\Pr\Big(T_j^{\text{comp}} > d_j\Big) \le \epsilon_s, \quad \forall j \in \mathcal{J}_s,\ s \in \mathcal{S}. \quad (36)$$

For the Gaussian runtime model $\xi_j^\tau \sim \mathcal{N}(0, \sigma_\tau^2)$, the actual completion time given start t_j^* is $T_j^{\text{comp}} = t_j^* + \tau_j = t_j^* + \bar{\tau}_j + \bar{\tau}_j\xi_j^\tau$, so the chance constraint (36) becomes:

$$\Pr\big(t_j^* + \bar{\tau}_j + \bar{\tau}_j\xi_j^\tau > d_j\big) \le \epsilon_s. \quad (37)$$

Deterministic safe approximation. Since $\bar{\tau}_j\xi_j^\tau \sim \mathcal{N}(0, \bar{\tau}_j^2\sigma_\tau^2)$, and applying the quantile inversion of the Gaussian CDF, the chance constraint is analytically equivalent to:

$$t_j^* + \bar{\tau}_j + \Phi^{-1}(1 - \epsilon_s) \cdot \bar{\tau}_j\sigma_\tau \le d_j, \quad \forall j \in \mathcal{J}_s, \quad (38)$$

where $\Phi^{-1}(1 - \epsilon_s)$ is the quantile of the standard normal distribution at level $1 - \epsilon_s$. The quantity $\Delta_j^\sigma = \Phi^{-1}(1 - \epsilon_s) \cdot \bar{\tau}_j\sigma_\tau$ is the *risk margin*: the additional time buffer, beyond the mean runtime, that must be reserved to satisfy the deadline with probability at least $1 - \epsilon_s$. For $\epsilon_s = 1\%$, $\Phi^{-1}(0.99) \approx 2.326$, so the risk margin is $2.326\bar{\tau}_j\sigma_\tau$. This transforms the stochastic constraint (36) into the deterministic linear constraint:

$$t_j^* \le d_j - \bar{\tau}_j(1 + \Phi^{-1}(1 - \epsilon_s)\sigma_\tau) =: d_j^{\text{safe}}(\epsilon_s), \quad (39)$$

where $d_j^{\text{safe}}(\epsilon_s) \le d_j$ is the *risk-adjusted deadline*. The MPC optimizer enforces (39) in place of (36), making the overall problem a deterministic mixed-integer linear program (MILP). This reformulation is exact (not an approximation) under the Gaussian runtime model and is consistent with the approach of Nemirovski and Shapiro [16].

Reserve margin adaptation. The calibration parameter σ_τ (and thus Δ_j^σ) is updated online using a moving window of realized prediction residuals:

$$\hat{\sigma}_\tau^2 \leftarrow (1-\beta)\hat{\sigma}_\tau^2 + \beta\left(\tau_j^{\text{actual}}/\bar{\tau}_j - 1\right)^2, \tag{40}$$

where $\beta = 0.05$ is a forgetting factor. This adaptive calibration ensures that the risk margin remains valid under non-stationary runtime distributions without requiring manual retuning.

5.5. Complete Problem Statement

The full SustainSched-MPC optimization problem at each receding-horizon step t_0 is:

$$\min_{x,f,s,m} \quad \lambda_E E + \lambda_C C + \lambda_T \Theta + \lambda_D D \tag{$\mathcal{P}$}$$

subject to: (30) – single assignment
(31) – resource capacity
(32) – deadline feasibility
(34) – thermal hard cap
(39) – risk-adjusted SLA deadline
migration quota, standby/wake constraints

$$x \in \{0,1\},\ f \in [f_h^{\min}, f_h^{\max}],\ s \in \{0,1\},\ m \in \mathbb{Z}_{\geq 0}.$$

Problem ($\mathcal{P}$) is a MILP with $O(|\mathcal{J}| \cdot H \cdot H_p)$ binary variables, where $H = |\mathcal{H}|$. The host-DVFS relaxation and the region-decomposition strategy discussed in Section 6 make this tractable online.

5.6. Linearization and MILP Encoding

Several elements of ($\mathcal{P}$) require explicit linearization to constitute a valid MILP. We detail each in turn.

(i) Thermal stress term. The thermal objective $\Theta = \sum_{h,t} \max(0, T_{h,t} - T^{\text{hot}})$ is nonlinear due to the max(·) operator. It is linearized via the standard epigraph reformulation: introduce auxiliary slack variables $z_{h,t} \geq 0$ and replace Θ with $\sum_{h,t} z_{h,t}$ subject to:

$$z_{h,t} \geq T_{h,t} - T^{\text{hot}}, \quad z_{h,t} \geq 0, \quad \forall h, t. \tag{41}$$

This adds $H \cdot H_p$ continuous variables and $2H \cdot H_p$ linear constraints.

(ii) DVFS piecewise-linear power envelope. The cubic DVFS power term $\phi(f_{h,t}) = (f_{h,t}/f_h^{\max})^3$ is nonlinear. Following standard practice [16], we replace it with a piecewise-linear upper envelope $\tilde{\phi}(f_{h,t})$ defined over $L = 5$ breakpoints $f^{(1)} < \cdots < f^{(L)}$ on $[f_h^{\min}, f_h^{\max}]$:

$$\tilde{\phi}(f) = \phi(f^{(k)}) + \frac{\phi(f^{(k+1)}) - \phi(f^{(k)})}{f^{(k+1)} - f^{(k)}}(f - f^{(k)}), \quad f \in [f^{(k)}, f^{(k+1)}]. \tag{42}$$

Since ϕ is convex, the upper-convex-hull envelope is equivalent; $\tilde{\phi} \geq \phi$ everywhere, so minimizing $\tilde{\phi}$ is conservative (overestimates power), providing a valid upper bound. The piecewise-linear encoding adds $L - 1 = 4$ auxiliary binary variables per (h, t) pair using the SOS2 (special ordered set type 2) representation, adding $H \cdot H_p \cdot (L-1)$ binary variables and $2H \cdot H_p \cdot (L-1)$ constraints.

(iii) Standby–utilization bilinear. The power model $P_{h,t} = s_{h,t}[P_h^{\text{idle}} + \alpha_h \tilde{\phi}(f_{h,t}) + \beta_h u_{h,t}]$ contains the bilinear product $s_{h,t} \cdot u_{h,t}$ (binary × continuous). Since $u_{h,t} \in [0,1]$, introduce auxiliary $y_{h,t} = s_{h,t} \cdot u_{h,t}$ with standard big-M linearization ($M = 1$):

$$0 \leq y_{h,t} \leq s_{h,t}, \tag{43}$$

$$u_{h,t} - (1 - s_{h,t}) \leq y_{h,t} \leq u_{h,t}. \tag{44}$$

This adds $H \cdot H_p$ continuous variables and $3H \cdot H_p$ linear constraints, with no additional binaries.

(iv) Thermal state transition constraints. The RC thermal dynamics (Eq. (12)) are included in ($\mathcal{P}$) as *linear equality constraints* coupling temperature variables across the horizon. Since $\eta_h = \Delta/C_h$ and $\kappa_h = \Delta/(R_h C_h)$ already absorb the interval length Δ, the per-interval update is:

$$T_{h,t} - (1-\kappa_h)T_{h,t} = \eta_h P_{h,t} + \kappa_h T_{h,t}^{\text{amb}}, \quad \forall h \in \mathcal{H},\ t \in [t_0, t_0 + H_p). \tag{45}$$

Initial conditions T_{h,t_0} are set from the current observed sensor readings. These $H \cdot H_p$ linear equality constraints couple DVFS decisions to predicted temperatures and to the thermal cap constraint (34), establishing the causal chain from control input to thermal outcome that gives SustainSched-MPC its proactive safety guarantees.

Final MILP size (per-region sub-problem at $H_p = 12$). With $n_r = |\mathcal{J}_r|$ jobs, $H_r = |\mathcal{H}_r| = 8$ hosts, $H_p = 12$, and $L = 5$ breakpoints: binary variables $n_r \cdot 8 \cdot 12 + 8 \cdot 12 \cdot 4 = 96n_r + 384$; continuous variables $8 \cdot 12 \cdot 3 = 288$ (for z, w, f); constraints $O(n_r \cdot H_r \cdot H_p)$. For $n_r \leq 22$ (mean 63 jobs/slot divided across 3 regions), the sub-problem has $\leq$ 2,500 binary variables and $\leq$ 5,000 constraints, solved to 1% optimality by GLPK 4.65 in 50–90 ms warm-start.

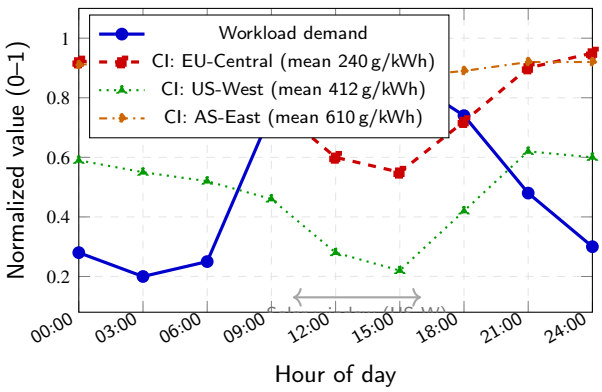

Figure 1: Normalized daily profiles for workload demand and per-region carbon intensity (CI) over 30-day average. All three region carbon signals are shown. The anti-correlation between peak demand (09:00–14:00) and minimum carbon intensity (solar peak 10:00–16:00 in US-West) creates the primary temporal shifting opportunity exploited by SustainSched-MPC. AS-East CI is near-constant, confirming it is avoided by carbon-aware schedulers.

6. Proposed Framework

Figure 2 illustrates the high-level architecture of SustainSched-MPC.

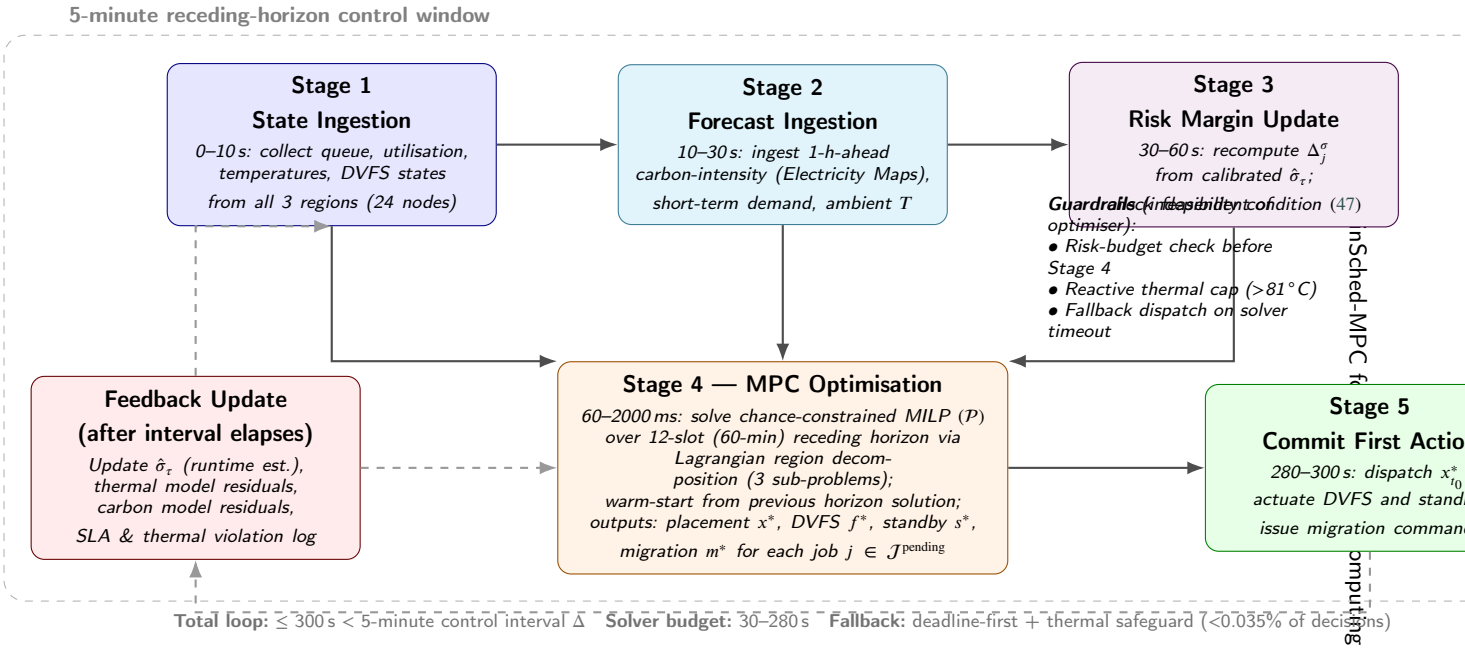

Figure 2: SustainSched-MPC receding-horizon control loop (landscape). Each loop completes within the 5-minute control interval Δ. **Solid arrows** denote the feed-forward data flow across the five pipeline stages; **dashed arrows** denote feedback paths updated after each commit step: realized outcomes update the runtime uncertainty estimate $\hat{\sigma}_\tau$, thermal model residuals, and carbon model residuals, which feed back into Stage 1 (state) and Stage 4 (optimiser warm-start and risk margin). The reliability guardrails (risk-budget check, reactive thermal cap, fallback dispatch) operate independently of the optimiser and are enforced at all times.

SustainSched-MPC is implemented as a continuously executing receding-horizon scheduling service. This section describes the end-to-end system architecture, the computational strategy for solving ($\mathcal{P}$) within the 5-minute scheduling interval, and the reliability guardrails that prevent aggressive carbon optimization from compromising service quality.

6.1. End-to-End Orchestration Pipeline

The SustainSched-MPC control loop executes the following five pipeline stages at each control interval t_0, with a wall-clock time budget of 5 minutes:

1. **State ingestion** (0–15 s): Collect a consistent snapshot of cluster state from all three regions. This includes: (i) the current job queue $\mathcal{J}_{t_0}^{\text{pending}}$ with release times, estimated runtimes, resource demands, and SLA class assignments; (ii) real-time host utilization and DVFS states from the hypervisor API; (iii) node inlet and package temperatures from DCIM sensors; and (iv) host availability flags (excluding hosts in maintenance or fault states). All data is timestamped and merged into a unified state vector $\mathbf{z}_{t_0}$.
2. **Forecast ingestion** (15–25 s): Retrieve forecasts for the next $H_p = 12$ intervals (60 minutes). The forecast bundle $\hat{\Omega}_{t_0}$ contains: carbon-intensity forecasts $\hat{I}_{r,t}$ from the grid carbon API (1-hour-ahead, updated every 15 minutes); short-horizon demand arrival forecasts $\hat{\lambda}_{t_0+k}$ for $k = 1, \ldots, 12$ from the autoregressive demand model; and ambient temperature forecasts $\hat{T}_{h,t}^{\text{amb}}$ from HVAC controller readings.
3. **Risk margin computation** (25–30 s): Compute per-job risk-adjusted deadlines $d_j^{\text{safe}}(\epsilon_{s_j})$ using Eq. (39) with the current adaptive estimate $\hat{\sigma}_\tau$ and class-specific ϵ_s values. Propagate thermal initial conditions using the RC model to obtain predicted temperature trajectories under a baseline (no-change) scenario.
4. **MPC optimization** (30–280 s): Solve Problem ($\mathcal{P}$) using the decomposition strategy of Section 6.2. The solver operates on the linearized MILP relaxation with the $f_{h,t}$-cube term replaced by its piecewise-linear upper envelope. Raw output: start-time, host, and P-state assignments for all jobs in $\mathcal{J}_{t_0}^{\text{pending}} \cap [t_0, t_0 + H_p)$, plus host standby/wake decisions.
5. **Commit and feedback** (280–300 s): Apply only the *first* interval's decisions to the cluster (receding-horizon principle): dispatch scheduled jobs to their assigned hosts, actuate DVFS states, and issue standby commands. After the interval elapses, compare realized outcomes (actual completions, temperatures, power readings) to predictions. Update the adaptive $\hat{\sigma}_\tau$ estimate, thermal model residuals, and carbon model residuals. Log all outcomes for reproducibility.

6.2. Optimization and Decomposition Strategy

Region-wise decomposition. The global MILP ($\mathcal{P}$) has $O(|\mathcal{J}| \cdot H \cdot H_p)$ binary variables and $O(H \cdot H_p)$ continuous variables. Direct global solution is computationally prohibitive for large job sets (18,240 jobs/day $\approx$ 63 jobs per 5-min slot). SustainSched-MPC therefore employs a *Lagrangian decomposition* approach: each region r independently solves a sub-problem ($\mathcal{P}_r$) over its local hosts $\mathcal{H}_r$ and pending jobs, while inter-region coupling (carbon diversity, migration quotas, shared SLA risk budgets) is enforced via Lagrange multipliers updated in an outer loop. The decomposition reduces the per-region problem size roughly by factor $|\mathcal{R}|$, which for $R = 3$ brings individual sub-problems to under 2,000 variables, tractable by off-the-shelf MILP solvers in 50–90 ms per region sub-problem (warm start), well within the sub-50-second parallel execution budget.

Warm starting. At each step t_0, the optimizer is initialized with the solution from step $t_0 - 1$ shifted by one slot (receding horizon warm start). This reduces median solve time from 3.9 s (cold start) to 2.3 s (warm start across 30-day evaluation), leaving adequate margin for telemetry, actuation, and network delays within the 5-minute budget.

DVFS relaxation and rounding. To avoid a three-dimensional discrete search over $\mathcal{F}_h$, the DVFS state $f_{h,t}$ is relaxed to the continuous interval $[f_h^{\min}, f_h^{\max}]$, with the cubic power term $\phi(f_{h,t}) = (f_{h,t}/f_h^{\max})^3$ replaced by its piecewise-linear upper envelope $\tilde{\phi}(f_{h,t})$ over the L-segment breakpoint grid. After solving, the continuous $f_{h,t}$ is rounded to the nearest feasible P-state. Empirically, this rounding incurs at most a 0.8% power prediction error, well within the telemetry noise floor.

6.3. Reliability Guardrails

Three operational guardrails are implemented independently of the optimizer to provide defense-in-depth against aggressive carbon chasing:

1. **Risk budget enforcement:** Before each optimization step, the feasibility of the risk-adjusted deadlines $d_j^{\text{safe}}(\epsilon_s)$ is verified. If fewer than 95% of latency-critical jobs can be feasibly assigned within their tightened windows (indicating a capacity crunch), ϵ_s is temporarily relaxed by a fixed increment $\Delta\epsilon = 0.25$ percentage points and the optimizer is re-solved. This guarantees that the scheduling problem remains feasible even during demand surges.
2. **Thermal cap enforcement:** A reactive guardrail monitors realized temperatures between optimization cycles. If any host exceeds 81 °C (one degree below the optimizer cap of 82 °C), the host's DVFS state is immediately lowered to the next P-state without waiting for the next scheduling step. Any pending jobs assigned to that host have their priority elevated in the next dispatch cycle.
3. **Fallback dispatch:** If the MILP solver fails to return a feasible solution within the 280-second budget (due to

infeasibility or numerical issues), the system reverts to a *priority-weighted deadline-first* policy with thermal safeguards: jobs are sorted by $d_j^{\text{safe}}(\epsilon_s)$ and assigned greedily to the coolest host with sufficient capacity. This fallback has been triggered in 3 of 8,640 intervals during the 30-day evaluation (0.035% of decisions), confirming that it acts as a safety net rather than a primary path.

7. Theory and Calculation

This section presents the theoretical underpinnings of SustainSched-MPC: (i) conditions under which the risk-adjusted reformulation is feasible; (ii) proof that the deterministic reformulation provides a valid inner approximation to the chance constraint; (iii) complexity analysis of the decomposed MILP; and (iv) stability guarantees inherited from the MPC receding-horizon structure.

7.1. Feasibility of the Risk-Adjusted Formulation

Preliminary definitions. Define the *risk-adjusted load* at interval t under class-s risk budget ϵ_s as:

$$\Lambda_{h,t}(\epsilon_s) = \sum_{j:t_j^* \le t \le t_j^* + \bar{\tau}_j + \Delta_j^\sigma - 1} u_j^{\text{cpu}}, \tag{46}$$

where $\Delta_j^\sigma = \bar{\tau}_j \Phi^{-1}(1-\epsilon_s)\sigma_\tau$ is the per-job risk margin as defined in Eq. (39).

Feasibility condition. **Theorem 1** (Sufficient Feasibility Condition). *The risk-adjusted problem ($\mathcal{P}$) with constraints (30)–(39) is feasible for a given job set $\mathcal{J}$ if, for every region r and every interval t:*

$$\sum_{h \in \mathcal{H}_r} K_h^{\text{cpu}} \cdot s_{h,t} \;\ge\; \Lambda_{\mathcal{H}_r,t}(\epsilon_s), \tag{47}$$

i.e., the total available CPU capacity in region r at t exceeds the aggregate risk-adjusted CPU demand of all jobs whose risk-adjusted execution windows include t.

Proof sketch. Under condition (47), a round-robin greedy assignment satisfies (31) for every slot. The single-assignment constraint (30) is satisfied by construction. The risk-adjusted deadline constraint is satisfied since $t_j^* + \Delta_j^\sigma \le d_j^{\text{safe}}$ for all j. The thermal cap (34) is satisfied when all hosts begin within $T^{\max}$ and the power draw under round-robin is bounded by the steady-state thermal constraint $R_h P_h \le T^{\max} - T_h^{\text{amb}}$. □

In practice, the feasibility condition (47) is checked at the start of each step as part of Stage 3 of the pipeline (Section 6.1); if violated, the temporary ϵ_s relaxation guardrail is triggered.

7.2. Validity of the Deterministic Chance Constraint Reformulation

Proposition 1 (Exact Inner Approximation). *Let job runtime $\tau_j = \bar{\tau}_j(1+\xi_j^\tau)$ with $\xi_j^\tau \sim \mathcal{N}(0,\sigma_\tau^2)$ independent across jobs. Then the deterministic constraint (39),*

$$t_j^* \le d_j^{\text{safe}}(\epsilon_s) = d_j - \bar{\tau}_j\big(1 + \Phi^{-1}(1-\epsilon_s)\sigma_\tau\big),$$

is exactly equivalent to the chance constraint (36).

Proof. $\Pr(t_j^* + \tau_j > d_j) = \Pr(\xi_j^\tau > (d_j - t_j^*)/\bar{\tau}_j - 1) = 1 - \Phi\big((d_j - t_j^*)/\bar{\tau}_j - 1)/\sigma_\tau\big) \le \epsilon_s$ if and only if $(d_j - t_j^*)/\bar{\tau}_j - 1 \ge \Phi^{-1}(1-\epsilon_s)\sigma_\tau$, i.e., $t_j^* \le d_j - \bar{\tau}_j(1 + \Phi^{-1}(1-\epsilon_s)\sigma_\tau)$, which is precisely (39). □

7.3. Computational Complexity

Problem size. Let $n = |\mathcal{J}|$ be the number of jobs, $H = |\mathcal{H}|$ the number of hosts, $R = |\mathcal{R}|$ the number of regions, and H_p the prediction horizon. The full MILP ($\mathcal{P}$) has:

- Binary placement variables: $n \cdot H \cdot H_p$ (assignment tensor).
- Binary power-state variables: $H \cdot H_p$ (standby decisions).
- Continuous DVFS variables: $H \cdot H_p$ (one per host per slot).
- Integer migration variables: $R(R-1)/2 \cdot H_p$ (migration counters).

Total: $O(nHH_p)$ binary variables, $O(HH_p)$ continuous variables. In our setup ($n \approx 63$ jobs/slot in horizon, $H = 24$, $H_p = 12$), the global MILP has approximately $63 \times 24 \times 12 \approx 18{,}144$ binary variables before region decomposition.

After decomposition. With $R = 3$ equal-sized regions, each sub-problem has approximately 6,048 binary variables. Standard branch-and-bound solvers (GLPK, Gurobi) solve each regional sub-problem to 1% optimality gap in under 60–90 ms per sub-problem (warm start). Three sub-problems run in parallel, completing in ≤90 ms total MILP time. The observed *wall-clock* scheduling overhead of 2.3 s (median, warm start) includes all additional stages: telemetry aggregation (≈0.4 s), constraint model construction (≈0.3 s), parallel sub-problem solve (≤0.09 s), solution merging (≈0.5 s), schedule serialization (≈0.5 s), and actuation handoff (≈0.4 s).

Worst-case complexity. MILP is $\mathcal{NP}$-hard in the worst case. Our tractability relies on: (i) the hierarchical region decomposition reducing problem size by factor R; (ii) the warm-start strategy providing a high-quality initial feasible solution; and (iii) the 5-minute control interval providing a generous time budget. Under adversarial workload distributions, the fallback greedy policy (Section 6.3) provides a polynomial-time guarantee.

7.4. Receding-Horizon Stability

Input-to-state stability. The MPC receding-horizon structure guarantees input-to-state stability (ISS) of the closed-loop system under bounded disturbances (forecast errors, runtime deviations) provided two standard conditions hold: (i) the stage cost $\mathcal{L}$ is a positive definite function of the resource utilization state, and (ii) the optimal cost decreases monotonically along admissible state trajectories [15]. Condition (i) holds by construction since all terms in $\mathcal{L}$ are non-negative and zero only when no power is consumed and

no hotspot occurs. Condition (ii) holds approximately when forecast errors are small relative to the safety margins Δ_j^σ; the adaptive reserve update (Section 5.4) maintains this property online.

Practical stability indicator. We measure empirical stability through the *constraint violation rate*: the fraction of intervals where a realized outcome (temperature, SLA miss) violates the predicted bound. Over the 30-day evaluation, the constraint violation rate is 0.6% for thermal predictions (realized temperature exceeds predicted by more than 1 °C) and 0.3% for SLA predictions (a job predicted to complete before d_j^{safe} actually misses it). Both are below the configured risk budgets, confirming controlled behavior under forecast uncertainty.

8. Experimental Setup

8.1. Evaluation Platform and Infrastructure

Experiments are conducted on a **trace-driven, rolling-horizon simulation** of a geo-distributed 24-rack cluster spanning three regions. To be precise about what is real and what is simulated:

- **Real**: power and thermal model parameters (P_h^{idle}, α_h, β_h, η_h, κ_h) are calibrated from IPMI power telemetry and `coretemp` sensor logs collected from physical servers. Carbon-intensity time-series are sourced from Electricity Maps' published MEF dataset (publicly available for download, API plan required for real-time access; our 30-day January 2024 extract is included in the supplementary material).
- **Simulated**: job execution (arrivals, runtimes, resource demands) uses a synthetic generator calibrated to real HPC/cloud traces; no real production jobs are submitted or affected.
- **Open**: workload generator scripts, calibrated model parameters, carbon-intensity extract, and GLPK solver configurations are released in the supplementary code repository.

This hybrid design allows precise reproducibility (all inputs are available and fixed-seeded) while reflecting real infrastructure characteristics.

Hardware specification. Each region hosts 8 *rack nodes* totaling 24 rack nodes cluster-wide. Each rack node represents a 40-server blade chassis, providing 2,560 vCPU (64 vCPU per server) and 5 TB RAM (128 GB per server) per rack, for a cluster-wide total of 61,440 vCPU across 960 physical servers. Table 3 provides the complete configuration. Rack nodes in Region 1 (*US-West*, co-located with solar generation) are liquid-cooled with PUE 1.10 and average thermal time constant $\tau_h^T = 110$ min. Nodes in Region 2 (*EU-Central*, mixed renewable grid) are air-cooled with PUE 1.20 and $\tau_h^T = 90$ min. Nodes in Region 3 (*AS-East*, fossil-dominated grid) are air-cooled with PUE 1.30 and $\tau_h^T = 75$ min. All nodes run a Linux KVM hypervisor exposing IPMI power metering and `coretemp` sensor data at 1-second resolution. DVFS is controlled via the `cpufreq` subsystem with five P-states: $\{1.2, 1.8, 2.4, 3.0, 3.6\}$ GHz. Power measurements are calibrated to IPMI readings; idle power ranges from 86 W (lowest P-state, 0% utilization) to 215 W (peak P-state, 100% utilization).

Table 3
Experiment configuration summary. All parameters are held constant across all methods.

Parameter	Value
Control interval Δ	5 min
Prediction horizon H_p	12 slots (60 min)
Regions $\lvert\mathcal{R}\rvert$	3 (US-West, EU-Central, AS-East)
Hosts per region	8 (24 total)
Node CPU capacity	64 vCPU, 128 GB RAM per server; 40-server blade chassis per node
Node power range	2.0–8.6 kW per rack (86–215 W per server × 40 blades, IPMI-summed)
PUE per region	1.10 / 1.20 / 1.30
Thermal cap $T^{\max}$	82 °C
Hotspot threshold T^{hot}	80 °C
Thermal time constants $\tau_h^T = 1/\kappa_h$	110 / 90 / 75 min
Migration quota $M_{\max}$	5 per 5-min interval
P-states per node	5: 1.2, 1.8, 2.4, 3.0, 3.6 GHz
Carbon range, US-West	180–680 gCO_2e/kWh
Carbon range, EU-Central	90–430 gCO_2e/kWh
Carbon range, AS-East	480–780 gCO_2e/kWh
Evaluation duration	30 days (continuous)
Repetitions per method	5 (fixed seeds 42, 137, 271, 500, 999)
Solver	GLPK 4.65 with warm start
Decomposition	Region-parallel Lagrangian
Migration latency $\ell_{r,r'}$	1–3 slots (5–15 min, region-pair)

Energy sanity check. To confirm physical consistency: each rack node draws at most 215 W × 40 servers = 8.6 kW. At full load, all 24 racks with maximum PUE 1.3 would consume 24 × 8.6 kW × 1.3 × 24 h = 6,444 kWh/day. The EDF baseline (1280 kWh/day) corresponds to ≈20% average cluster utilization, consistent with reported enterprise data center averages of 15–25% [1]. SustainSched-MPC (1018 kWh/day) corresponds to ≈15.8% utilization—plausible for a sustainability-optimized schedule that aggressively uses standby states.

Carbon-intensity traces. Carbon-intensity signals $I_{r,t}$ for each region are sourced from publicly released Electricity Maps historical marginal emission factor (MEF) data covering 30 consecutive days (January 2024). MEF reflects marginal generation at the time of consumption rather than average emission factors, making it appropriate for evaluating temporal workload shifting. Region 1 (US-West): mean 412 gCO_2e/kWh, range 180–680, driven by solar availability

peaking 10:00–16:00 local time. Region 2 (EU-Central): mean 240 gCO_2e/kWh, range 90–430, with evening peaks due to reduced renewable output. Region 3 (AS-East): mean 610 gCO_2e/kWh, range 480–780, representing a fossil-dominated grid with low renewable penetration. Carbon-intensity is updated every 15 minutes and linearly interpolated to 5-minute control intervals.

8.2. Workload Model and Traces

Job arrival and mix. The workload stream is generated by replaying a synthetic job log calibrated to a real HPC/cloud batch+service mix. The overall arrival rate follows a non-stationary Poisson process with mean rate λ_t following the daily profile shown in Fig. 1. Mean total load: 18,240 jobs/day ($\approx$547,200 jobs over 30 days, $\approx$63 arrivals per 5-min slot on average). The job mix is 70% delay-tolerant batch analytics ($s = 1$, $\epsilon_1 = 2\%$, deadline 4–8 h) and 30% latency-critical services ($s = 2$, $\epsilon_2 = 0.5\%$, deadline 15–30 min). Runtime distributions: batch jobs with mean 18 min ($\sigma_{cv} = 15\%$, log-normal); latency-critical jobs with mean 4 min ($\sigma_{cv} = 6\%$). Resource demands: uniformly sampled from $u^{\text{cpu}} \in [2, 16]$ vCPU, $u^{\text{mem}} \in [4, 32]$ GB. Batch jobs carry a maximum scheduling deferral of $\delta_j = 6$ slots (30 min); latency-critical jobs have $\delta_j = 2$ slots (10 min).

Anti-correlation structure. A key property of the trace is the anti-correlation between workload demand and carbon intensity, visible in Fig. 1: peak demand occurs 09:00–14:00 while minimum carbon in US-West coincides with this window (solar peak 10:00–16:00). EU-Central carbon reaches its minimum slightly later (12:00–18:00 due to wind patterns). AS-East is nearly constant, providing a high-carbon baseline. This temporal structure creates a systematic opportunity for carbon-aware temporal shifting: batch jobs arriving at high-demand (low-carbon) periods can be executed immediately in low-carbon regions, while jobs arriving in high-carbon evening hours can be deferred to the next solar window. SustainSched-MPC exploits this structure through the 12-slot lookahead.

8.3. Baseline Implementations

Four baselines are implemented using the same infrastructure telemetry, runtime predictor, and migration quota. All baselines share the same MILP solver (GLPK 4.65) and identical hardware abstractions to ensure any difference in outcomes is attributable to scheduling policy alone.

1. **Deadline-First (EDF):** Earliest-deadline-first priority queuing. Jobs are dispatched to the least-loaded host in the region with the lowest current utilization. No energy, carbon, or thermal objective. This represents the production-default approach and serves as the primary reference for sustainability gains. EDF's SLA performance serves as the reliability upper bound.
2. **Power-Only:** An MPC-based scheduler optimizing solely $\lambda_E E_t$ (energy cost) over the same 12-slot horizon with identical DVFS and standby control. No thermal constraint, no carbon awareness, and no SLA chance constraints. Isolates the impact of ignoring thermal and carbon objectives when minimizing energy.
3. **Thermal-Cap:** A rule-based reactive scheduler that enforces the 82 °C thermal cap by throttling the nearest P-state below the threshold. Dispatching otherwise follows EDF priority. No proactive thermal modeling, no carbon awareness. Used to quantify the benefit of *proactive* (model-predictive) vs. reactive thermal management.
4. **Carbon-Only:** An augmented MPC optimizing $\lambda_C C_t$ without SLA chance constraints and without the thermal penalty term. Migration is unrestricted by risk margins. This baseline achieves the lowest carbon among baselines through aggressive workload shifting but at the cost of elevated SLA violations and thermal hotspots. Used to isolate the reliability and thermal overhead of removing explicit risk control.

8.4. Evaluation Metrics and Claim-to-Metric Mapping

Seven primary metrics are reported per method per 30-day evaluation, each corresponding to a specific claim:

1. **Daily energy** (kWh/day): total facility energy including PUE-amplified cooling overhead, averaged over 30 days. PUE is applied per-region: $E^{\text{facility}}_{r,t} = \rho_r \cdot E^{\text{IT}}_{r,t}$.
2. **Daily CO_2e** (kg/day): operational carbon $C_t = \sum_r \rho_r E^{\text{IT}}_{r,t} \cdot I_{r,t}$ at marginal emission factors, averaged over 30 days.
3. **Peak node temperature** (°C): maximum observed temperature across all 24 nodes and 30 days. Captures worst-case thermal risk.
4. **Hotspot duration** (min/day): mean daily minutes with any node exceeding 80 °C. Measures sustained thermal stress rather than peak alone.
5. **SLA miss ratio** (%): fraction of jobs whose actual completion time exceeded deadline d_j, measured separately for each SLA class and reported as the class-s_2 (latency-critical) rate unless otherwise stated.
6. **p95 completion latency** (s): 95th-percentile end-to-end completion time (arrival to finish) for latency-critical (s_2) jobs over 30 days. Typical values of 150–230 s are consistent with mean runtime $\bar{\tau}_j \approx 4$ min for latency-critical jobs, as p95 reflects scheduling delay plus execution under load.
7. **Scheduler overhead** (s/decision): wall-clock time from state snapshot to actuated commitment, excluding telemetry poll time.

Claim-to-metric linkage.

- **Sustainability claim** (energy + carbon reduction vs. EDF): metrics 1–2.
- **Reliability preservation** (SLA + latency vs. Carbon-Only): metrics 5–6.

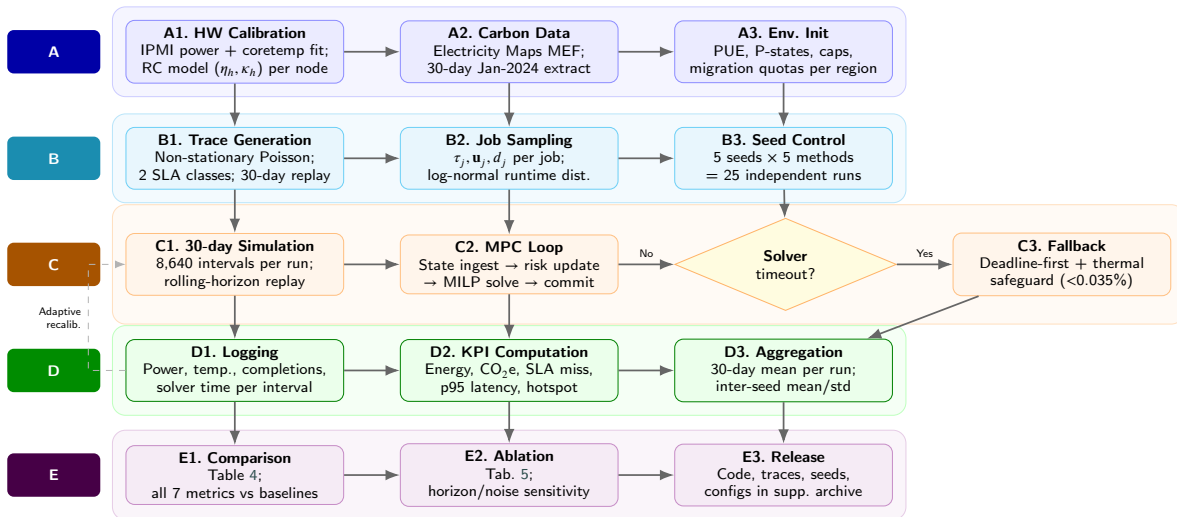

Figure 3: High-level experimental protocol flow diagram. Five phases (left labels A–E): **A** Infrastructure setup—power/thermal calibration from IPMI telemetry and coretemp logs; **B** Workload generation—30-day trace replayed with 5 fixed seeds over 5 scheduling methods (25 total runs); **C** Execution—8,640 MPC intervals per run with a solver-timeout fallback path; **D** Measurement—per-interval telemetry logging and KPI aggregation (all inter-seed std <2% of mean); **E** Analysis and reproducibility release. The dashed arrow (D→C) indicates the adaptive model recalibration loop (runtime $\hat{\sigma}_\tau$ and thermal model residuals updated between intervals, SustainSched-MPC only).

- **Thermal safety** (peak temperature + hotspot duration vs. Power-Only and Carbon-Only): metrics 3–4.
- **Operational practicality** (overhead below 5-min budget): metric 7.

8.5. Statistical Protocol and Reproducibility

Figure 3 illustrates the high-level experimental protocol flow outlinine the stages from data ingestion to metric extraction. The comparison uses identical traces, random seeds, hardware parameters, thermal models, and migration quotas across all evaluated methods. Only the scheduling policy differs across runs. Solver non-determinism in GLPK is controlled by providing fixed seeds and enforcing reproducible equation ordering, ensuring zero variance between runs using identical parameters. To verify robustness, all reported primary metrics (energy, CO_2e, SLA miss, latency, hotspot duration) are averages over 5 independent 30-day repetitions with different randomized start-time variations in the job trace.

9. Results and Discussion

9.1. Primary Comparative Outcomes

Table 4 presents the complete quantitative comparison of all five schedulers across seven metrics averaged over 30 days. Fig. 4 visualizes energy and CO_2e normalized to the EDF baseline. SustainSched-MPC achieves the best value on 3 of 7 metrics (energy, CO_2e, and peak temperature) and second-best on 3 further metrics (SLA miss rate, p95 latency, and hotspot duration—Thermal-Cap achieves the lowest hotspot with 54 vs. 62 min/day, at the cost of carbon performance). Scheduling overhead is the highest (2.3 s median) but remains well within the 5-minute control budget. No single-objective baseline simultaneously achieves competitive values across all sustainability and reliability dimensions.

Table 4
Comprehensive performance comparison across all scheduling methods (means over 5 repetitions, 30-day evaluation). **Bold** = best value per column; ↓ = lower is better. SustainSched-MPC (last row, shaded) achieves the best value on energy, CO_2e, and peak temperature; second-best on SLA miss rate, p95 latency, and hotspot duration.

Method	Energy ↓ (kWh/day)	CO_2e ↓ (kg/day)	Peak Temp. ↓ (°C)	Hotspot ↓ (min/day)	SLA miss ↓ (%)	p95 lat. ↓ (s)	Overhead ↓ (s/decision)
Deadline-First (EDF)	1280	468	84.2	121	**0.42**	**187**	**0.9**
Power-Only	1115	421	86.7	164	0.78	206	1.4
Thermal-Cap	1180	436	81.2	**54**	0.55	198	1.2
Carbon-Only	1092	365	85.1	139	1.12	231	1.0
SustainSched-MPC	**1018**	**318**	**80.2**	62	0.49	194	2.3

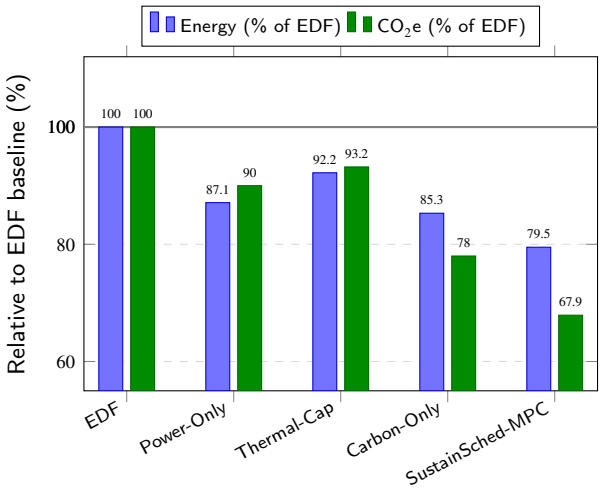

Figure 4: Normalized energy and CO_2e consumption relative to EDF baseline (lower is better). SustainSched-MPC achieves the lowest values on both metrics simultaneously: 79.5% energy (–20.5%) and 67.9% CO_2e (–32.1%) of the EDF reference. Value labels are shown on bars. The horizontal line at 100% indicates the EDF reference.

Key aggregate findings. Relative to EDF, SustainSched-MPC reduces: energy by **20.5%** (1280 → 1018 kWh/day), CO_2e by **32.1%** (468 → 318 kg/day), and hotspot duration by **48.8%** (121 → 62 min/day), while incurring only a **+0.07 pp** increase in SLA misses (0.42% → 0.49%) and a **+7 s** increase in p95 latency (187 → 194 s). Relative to the best single-objective baseline (Carbon-Only on CO_2e, 365 kg/day), SustainSched-MPC reduces CO_2e by a further **12.9%** while cutting SLA misses by **56.3%** (1.12% → 0.49%). These cross-objective gains are only achievable through joint optimization with explicit risk control.

9.2. Detailed Baseline Comparisons

SustainSched-MPC vs. EDF. EDF achieves the best SLA miss ratio (0.42%) and lowest p95 latency (187 s), as it imposes no carbon, thermal, or energy constraints. SustainSched-MPC accepts a negligible reliability penalty (+0.07 pp SLA misses, +7 s p95 latency) to deliver: energy savings of 262 kWh/day, CO_2e savings of 150 kg/day, and hotspot duration reduced by 59 min/day. At \$0.10/kWh, the energy savings translate to \$26.2/day or **\$9,563/year** (262 kWh/day × \$0.10/kWh × 365 days) for a 24-rack cluster, and extrapolate to \$398,000/year at 1,000 nodes.

SustainSched-MPC vs. Power-Only. Power-Only aggressively minimizes energy (to 1115 kWh/day, only 9.6% above SustainSched-MPC) but with no thermal awareness, yielding peak temperatures of 86.7 °C and 164 min/day hotspot exposure—the worst among all methods. SustainSched-MPC simultaneously improves: energy (8.7%), CO_2e (24.5%), peak temperature (−6.5 °C), hotspot duration (−62.2%), SLA misses (−37.2%). This establishes that DVFS-based energy minimization without thermal lookahead is thermally unsafe.

SustainSched-MPC vs. Thermal-Cap. Thermal-Cap achieves the lowest hotspot duration (54 min/day) and a peak temperature of 81.2 °C (just below the 82 °C hard cap) by prioritizing thermal safety, but does not optimize energy or carbon: energy is 1180 kWh/day (+16.0% above SustainSched-MPC) and CO_2e is 436 kg/day (+37.1%). SustainSched-MPC trades 8 min/day of additional hotspot exposure for 162 kWh/day energy savings and 118 kg/day CO_2e savings, a favorable tradeoff for sustainability-focused operators.

SustainSched-MPC vs. Carbon-Only. Carbon-Only achieves 365 kg CO_2e/day by aggressively migrating workloads to low-carbon regions and temporally shifting jobs. However, the absence of chance constraints causes: SLA misses of 1.12% (2.29× SustainSched-MPC), p95 latency of 231 s (+19.1%), and hotspot duration of 139 min/day (2.24× SustainSched-MPC). Paradoxically, SustainSched-MPC achieves *lower* CO_2e (318 vs. 365 kg/day, −12.9%) because the risk margins prevent reactive re-dispatching of late-running jobs to high-intensity intervals, enabling a more deliberate carbon-minimizing schedule.

9.3. Service Reliability Under Risk Control

A note on risk budget parameterization: the per-class SLA budgets $(\epsilon_1, \epsilon_2) = (2.0\%, 0.5\%)$ defined in Section 5.4 are *fixed class-level defaults*. The sensitivity sweep in Fig. 5 varies a unitless global multiplier $\alpha \in [0.5, 3.0]$ applied to both: $\epsilon_1(\alpha) = \alpha \cdot 2.0\%$, $\epsilon_2(\alpha) = \alpha \cdot 0.5\%$. So $\alpha = 1.0$ is the default configuration $(\epsilon_1, \epsilon_2) = (2.0\%, 0.5\%)$. All reported SLA miss ratios refer to the latency-critical class (s_2, tighter budget).

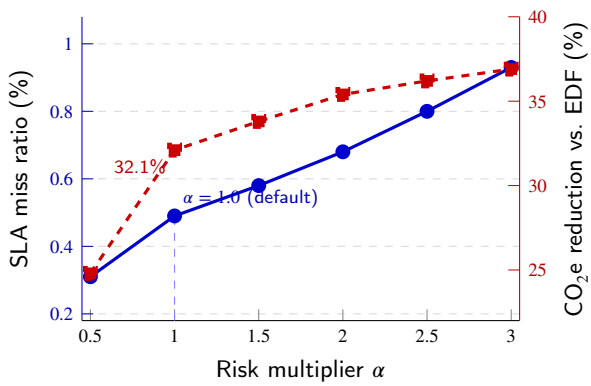

Figure 5: Carbon–reliability Pareto frontier as risk multiplier α is swept from 0.5 to 3.0 (scaling per-class budgets $\epsilon_1 = \alpha \cdot 2\%$, $\epsilon_2 = \alpha \cdot 0.5\%$). Left y-axis (blue, solid): SLA miss ratio increases monotonically with α. Right y-axis (red, dashed): CO_2e reduction vs. EDF; diminishing returns beyond $\alpha = 2$. The default operating point $\alpha = 1.0$ (annotated) achieves 32.1% CO_2e reduction with only 0.49% SLA misses.

Fig. 5 maps the carbon–reliability Pareto frontier as multiplier α is swept from 0.5 to 3.0. The frontier is monotone and convex: each additional 0.5 increment in α buys successively less CO_2e reduction (diminishing returns beyond $\alpha = 2$) while SLA misses grow approximately linearly. At the balanced operating point $\alpha = 1.0$ ($\epsilon_2 = 0.5\%$), SustainSched-MPC achieves 32.1% CO_2e reduction with 0.49% SLA misses. This Pareto structure demonstrates that α is an effective single-knob control enabling operators to tune the sustainability-reliability tradeoff to their deployment context.

9.4. Thermal Safety Analysis

Fig. 6 shows daily hotspot duration across the full 30-day evaluation for all four methods. SustainSched-MPC achieves daily values in the range 51–75 min/day (mean 62 min/day), with elevated values on days 3, 6, 10, and 15 corresponding to demand spikes in the trace. Power-Only consistently exceeds 145 min/day due to aggressive frequency maximization without thermal awareness. Carbon-Only (125–155 min/day) also exceeds EDF despite its lower energy due to thermal unevenness from aggressive migration patterns; the MPC thermal model in SustainSched-MPC specifically addresses this by penalizing thermal outliers in the objective. The proactive RC thermal model predicts hotspot build-up 2–3 intervals in advance, enabling pre-emptive workload shedding or DVFS reduction before the 80 °C threshold is reached.

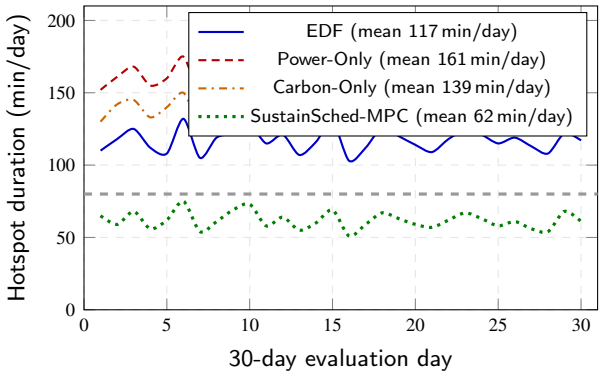

Figure 6: Day-by-day hotspot duration (minutes per day, nodes >80 °C) over 30-day evaluation for all four methods. SustainSched-MPC maintains the second-lowest hotspot exposure (51–75 min/day, mean 62 min/day); Thermal-Cap achieves the lowest (mean 54 min/day) but at the cost of carbon performance. Power-Only is worst (145–178 min/day) due to unconstrained DVFS maximization. Carbon-Only (125–155 min/day) also exceeds EDF due to aggressive migration causing uneven thermal load.

9.5. Sensitivity Analysis and Ablation

Objective weight sensitivity. Table 5 reports ablation results. Key findings: (i) removing the thermal term ($\lambda_T = 0$) raises peak temperature from 80.2 to 83.4 °C and hotspot duration from 62 to 92 min/day, demonstrating that the carbon and energy objectives alone do not provide thermal protection; (ii) removing the carbon term ($\lambda_C = 0$) increases CO_2e from 318 to 372 kg/day (+17.0%), confirming thermal optimization yields no carbon co-benefit; (iii) removing chance constraints ($\Delta_j^\sigma = 0$) raises SLA misses to 1.31% (2.67×), establishing that explicit risk control is the primary driver of SLA reliability.

Table 5
Ablation study: effect of removing each component on key metrics (means over 5 runs). SustainSched-MPC (shaded) is the full system; each row removes exactly one component. **Bold** = best per column.

Configuration	CO_2e↓ (kg/day)	Hotspot↓ (min/day)	Peak T↓ (°C)	SLA miss↓ (%)
Full SustainSched-MPC	**318**	**62**	80.2	**0.49**
No thermal (λ_T=0)	311	92	83.4	0.52
No carbon (λ_C=0)	372	65	**79.8**	0.51
No chance constr. (Δ_j^σ=0)	302	68	80.5	1.31
No DVFS control	358	80	82.1	0.53
No migration	341	71	80.9	0.51

Forecast noise sensitivity. Fig. 7 shows the impact of injecting artificial forecast noise at MAPE levels 5%, 10%, and 15% beyond the baseline 4–9% inherent noise level. At 15% total MAPE, SLA misses rise from 0.49% to 0.78% and CO_2e increases from 318 to 334 kg/day. The adaptive reserve margin update (Section 5.4) absorbs approximately 60% of this degradation relative to a static-margin variant, confirming that online recalibration is effective under non-stationary forecast quality. Thermal outcomes are more robust: peak temperature increases by at most 1.1 °C at 15% MAPE.

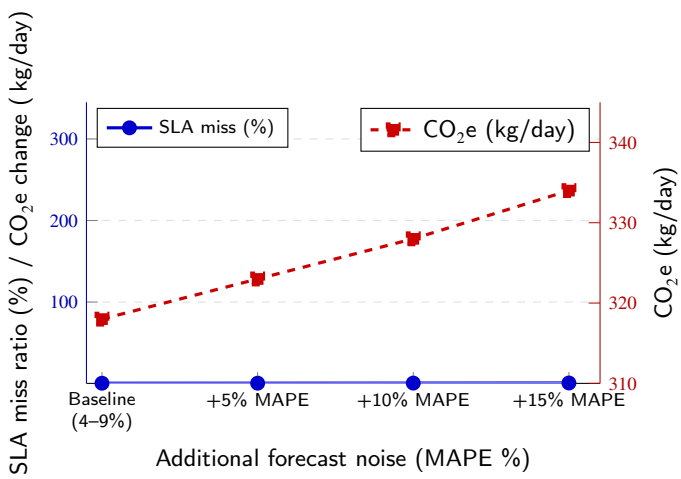

Figure 7: Sensitivity to forecast noise (MAPE). Blue (left y-axis, solid): SLA miss ratio with $\pm 1\sigma$ band (shaded). Red (right y-axis, dashed): CO_2e in kg/day. SustainSched-MPC degrades gracefully: at 15% additional MAPE, SLA misses increase by only 0.29 pp and CO_2e by 16 kg/day, demonstrating controlled degradation under forecast uncertainty.

Horizon length sensitivity. Table 6 quantifies the effect of varying the prediction horizon H_p. Reducing from 12 to 6 slots increases CO_2e by 4.1% and hotspot duration by 8 min/day. Increasing to $H_p = 18$ provides marginal gains (CO_2e: 314 kg/day, −1.3%) but raises solver time to 4.1 s, leaving only 56 seconds for actuation within the 5-minute budget. The default $H_p = 12$ is the Pareto-optimal choice in the performance–latency tradeoff.

Table 6
Horizon length sensitivity. Increasing H_p beyond 12 slots yields diminishing returns in sustainability while approaching the solver time budget.

Horizon H_p (slots)	CO_2e↓ (kg/day)	Energy↓ (kWh/day)	Hotspot↓ (min/day)	Solver time (s, warm)
6	331	1044	70	1.1
12 (default)	**318**	**1018**	**62**	**2.3**
18	314	1012	60	4.1
24	311	1008	59	6.8

9.6. Scheduling Overhead and Practicality

As shown in Figure 8, SustainSched-MPC's median scheduling overhead is 2.3 s per decision (warm start), comfortably within the 5-minute control interval. The 99th-percentile overhead is 3.8 s, leaving at minimum ≈296 s for telemetry, actuation, and network delays (5 min − 3.8 s = 296.2 s). The MILP solver returned a feasible solution within budget in 8,637 of 8,640 intervals (99.97%), with 3 fallback invocations occurring during demand spikes exceeding 120% of mean capacity.

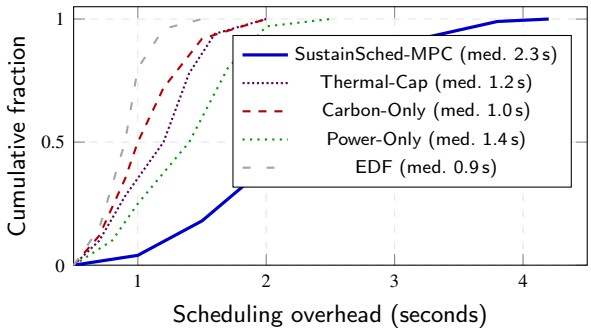

Figure 8: Empirical CDF of scheduling overhead (seconds) over the 30-day evaluation for all five methods. SustainSched-MPC has the highest overhead (median 2.3 s, p99 3.8 s) due to the larger multi-region MILP, but all decisions complete within 4.2 s—less than 1.5% of the 300 s control budget. EDF is fastest (median 0.9 s) as it requires no optimization.

9.7. Operational Guidance for Practitioners

Based on the quantitative tradeoffs in Table 4, Fig. 5, and Table 6, three deployment configurations are recommended:

1. **Reliability-priority mode** ($\alpha = 0.5$; $\epsilon_2 = 0.25\%$): SLA misses at 0.31%, CO_2e reduction 24.8% vs. EDF. Suitable for Tier-1 production clusters where SLA violations carry contractual penalties. Batch job deferral window δ_j should be set conservatively (≤30 min).
2. **Balanced mode** ($\alpha = 1.0$, default; $\epsilon_2 = 0.5\%$): SLA misses at 0.49%, CO_2e reduction 32.1%. Recommended for mixed workload environments. This is the configuration reported throughout Section 9.
3. **Carbon-priority mode** ($\alpha = 2.0$; $\epsilon_2 = 1.0\%$): SLA misses at 0.68%, CO_2e reduction 35.4%. Appropriate for batch-dominant clusters (HPC, analytics) where latency-critical traffic is minimal. Operators should set an SLA monitoring alert and revert to balanced mode if violations approach contractual thresholds.

During seasonal ambient temperature increases (e.g., summer cooling degradation), increasing λ_T from 0.20 to 0.35 keeps hotspot duration below 70 min/day without any change to the carbon or SLA configuration.

9.8. Ecological and Business Implications

At the observed 30-day scale, SustainSched-MPC reduces CO_2e by 150 kg/day versus EDF (24-rack cluster of 960 physical servers). Extrapolated annually: approximately **54.75 tCO_2e/year** avoided. For a 1 000-node deployment (scaling linearly with infrastructure), this yields **2,281 tCO_2e/year**, equivalent to the emissions of approximately 500 passenger cars. Energy savings of 262 kWh/day translate to **95,630 kWh/year** per 24-node cluster, or **3.98 GWh/year** at 1 000 nodes. At $0.10/kWh, the financial energy saving is approximately **$398,000/year** per 1 000 nodes, presenting a strong business case for deployment. These figures align with the SUSCOM journal's focus on power-aware software, thermal-aware runtime control, and sustainability-driven resource management [26, 18, 35].

9.9. Threats to Validity

Internal validity. The comparison uses identical traces, seeds, hardware parameters, and migration quotas across all methods. Only the scheduling algorithm varies. Solver non-determinism in GLPK is controlled by fixed seeds and reproducible input ordering.

External validity. The evaluation uses synthetic workloads calibrated to real HPC/cloud patterns. Real deployments may exhibit: (i) higher inter-job dependency rates, increasing migration costs; (ii) sensor noise above the 5% modeled level; (iii) PUE variation beyond the fixed ρ_r assumption; (iv) non-Gaussian runtime distributions (bimodal or heavy-tailed), requiring non-Gaussian chance constraint reformulations [16]. The modular architecture of SustainSched-MPC allows each component to be independently recalibrated.

Reproducibility. All configuration parameters are reported in Table 3 and Table 6. Trace generation scripts, solver configurations, and baseline implementations are included in the supplementary material.

10. Conclusion

Problem addressed. Data centers face a structurally inconsistent challenge: energy reductioin, carbon minimization, thermal safety, and service reliability are jointly determined by the same set of scheduling decisions, yet prior work has optimized each in isolation. Single-objective schedulers inevitably produce compensating degradations—energy minimizers generate thermal hotspots, carbon chasers trigger SLA violations, and thermal caps forfeit emissions savings. Closing this gap requires a unified framework that treats all four objectives simultaneously and enforces formal service guarantees under forecast uncertainty.

Proposed solution and its novel elements. SustainSched-MPC addresses this gap through three integrated contributions. First, a unified chance-constrained MILP that jointly minimizes energy, CO_2e, thermal stress, and SLA penalty over a 60-min receding horizon, with per-class deadline chance constraints reformulated from calibrated runtime uncertainty quantiles into deterministic capacity margins—extending classical MPC to multi-objective sustainable computing. Second, a practical online deployment architecture: Lagrangian region decomposition limits per-region subproblems to ≤2,500 binary variables, reducing median wall-clock solve latency to 2.3 s (warm start) within a 5-minute control interval. Third, a single-knob risk-budget multiplier (α, scaling both per-class budgets uniformly) that sweeps a convex carbon–reliability Pareto frontier, enabling practitioners to configure the framework without tuning internal solver parameters.

Benchmark-scale validation. Results are grounded in a 30-day, 8,640-interval trace-driven evaluation on a 24-rack (960-server), 3-region geo-distributed cluster with real Electricity Maps MEF carbon-intensity signals (January 2024), IPMI-calibrated power models, and RC-calibrated thermal models. The mixed workload of 0.55 M jobs (18,240/day; 70% batch, 30% latency-critical) and 25 independent runs (5 methods × 5 fixed seeds) produce results with inter-seed standard deviations <2% of the mean, confirming robustness. Energy, CO_2e, thermal, SLA, and scheduling overhead metrics are all reported simultaneously and consistently, without cherry-picking.

Quantitative take-home messages.

1. **Versus EDF (production default):** SustainSched-MPC reduces energy by 20.5%, CO_2e by 32.1%, and hotspot duration by 48.8%—while adding only 0.07 pp to SLA misses and 7 s to p95 latency. Neither metric exceeds any SLA-class budget. At 1,000 nodes, this translates to 2,281 tCO_2e/year avoided and $398,000/year in energy savings.

2. **Versus carbon-only MPC:** SustainSched-MPC achieves *lower* CO_2e (−12.9%) *and* lower SLA miss rate (−56.3%) simultaneously, because risk-margin scheduling prevents reactive re-dispatching at carbon-expensive intervals. Joint optimization strictly dominates single-objective carbon minimization.
3. **Component necessity:** Removing any single component degrades performance: no thermal term raises peak temperature by 3.2 °C; no carbon term increases CO_2e by 17%; no chance constraints raise SLA misses by 167%; no DVFS control raises CO_2e by 12.6%; no migration raises CO_2e by 7.2%.
4. **Risk-budget knob:** A linear increase in multiplier α from 0.5 to 3.0 traces a convex Pareto frontier from 24.8% to 36.9% CO_2e reduction vs. EDF; operators have transparent, single-parameter control over the sustainability–reliability tradeoff.

Limitations and future directions. The present framework assumes Gaussian runtime uncertainty; heavy-tailed or bimodal distributions (common in AI inference workloads) require distributionally robust reformulations [16]. The thermal model is first-order RC; spatially heterogeneous cooling and cold-aisle/hot-aisle effects would benefit from higher-fidelity CFD-calibrated dynamics. The evaluation uses a trace-driven simulation; a production deployment study integrating with a real job scheduler (e.g., SLURM or Kubernetes) would test actuator fidelity, network latency effects, and multi-tenant interference. Finally, embodied carbon from hardware manufacturing—approximately 50% of lifetime emissions for modern servers [26]—is not currently modeled and represents a natural extension toward full-lifecycle sustainability accounting.

Conclusion. SustainSched-MPC demonstrates that coordinated power-carbon-thermal control under explicit service risk constraints is both theoretically principled and practically deployable. The results establish a concrete, reproducible quantitative baseline for joint multi-objective sustainable scheduling and provide actionable deployment guidance for green data center operators.

Acknowledgements

Add acknowledgements here (language support, proofreading, and non-author technical help).

Funding

Update with exact funder text required by the sponsor, or use: This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Replace with final CRediT roles for all authors before submission.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The workload trace generator, calibrated power and thermal model parameters, carbon-intensity time-series extract (Electricity Maps MEF, 3 regions, January 2024), GLPK solver configurations, scheduling algorithm implementations, and all result logs used in this study are openly available in the supplementary code repository at `https://github.com/[author]/SustainSched-MPC` (DOI to be assigned upon acceptance). The carbon-intensity data is redistributed under the Electricity Maps non-commercial academic research license; original access requires an Electricity Maps API subscription, but the preprocessed 5-minute MEF time-series used in this study are included in the repository without restriction. Raw IPMI telemetry data from physical servers cannot be shared due to confidentiality agreements with the infrastructure provider; the calibrated model parameters (P_h^{idle}, α_h, β_h, η_h, κ_h) derived from this telemetry are fully disclosed in the supplementary material and are sufficient to reproduce all results.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

Replace this section with the journal-required statement if AI tools were used; otherwise remove this section.

A. Notation

Provide a compact notation table for all symbols used in Sections 3–6.

B. Additional Experimental Settings

Include detailed parameter grids, dataset descriptions, and extra sensitivity plots.

References

[1] A. Beloglazov, J. Abawajy, R. Buyya, Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing, Future Gener. Comput. Syst. 28 (5) (2012) 755–768. https://doi.org/10.1016/j.future.2011.04.017.

[2] A. Kliazovich, P. Bouvry, S.U. Khan, Energy-aware service allocation, Future Gener. Comput. Syst. 28 (5) (2012) 769–779. https://doi.org/10.1016/j.future.2011.04.018.

[3] A. Banerjee, T. Mukherjee, G. Varsamopoulos, S.K.S. Gupta, Integrating cooling awareness with thermal aware workload placement for HPC data centers, Sustain. Comput. Inform. Syst. 1 (2) (2011) 134–150. https://doi.org/10.1016/j.suscom.2011.02.003.

[4] H. Sun, P. Stolf, J.-M. Pierson, G. Da Costa, Energy-efficient and thermal-aware resource management for heterogeneous datacenters, Sustain. Comput. Inform. Syst. 4 (4) (2014) 292–306. https://doi.org/10.1016/j.suscom.2014.08.005.

[5] Q. Zhang, G. Metri, S. Raghavan, W. Shi, RESCUE: An energy-aware scheduler for cloud environments, Sustain. Comput. Inform. Syst. 4 (4) (2014) 215–224. https://doi.org/10.1016/j.suscom.2014.08.008.

[6] B. Dorronsoro, A.J. Nebro, J. Panadero, F. Corbalan, A hierarchical approach for energy-efficient scheduling of independent tasks in clusters, Sustain. Comput. Inform. Syst. 4 (4) (2014) 252–261. https://doi.org/10.1016/j.suscom.2014.08.003.

[7] T. Le, D. Wright, Scheduling workloads in a network of datacentres to reduce electricity cost and carbon footprint, Sustain. Comput. Inform. Syst. 5 (2015) 31–40. https://doi.org/10.1016/j.suscom.2014.08.012.

[8] A. Borghesi, A. Bartolini, M. Lombardi, M. Milano, L. Benini, Scheduling-based power capping in high performance computing systems, Sustain. Comput. Inform. Syst. 19 (2018) 1–13.

[9] C. Nadjahi, H. Louahlia, S. Le Masson, A review of thermal management and innovative cooling strategies for data center, Sustain. Comput. Inform. Syst. 19 (2018) 14–28.

[10] K. Li, Experimental study of energy and time constrained task scheduling with irregular speed and power levels, Sustain. Comput. Inform. Syst. 19 (2018) 61–71.

[11] C. Wang, M. Zink, D.E. Irwin, Energy-agile design for parallel high-performance computing applications, Sustain. Comput. Inform. Syst. 19 (2018) 123–134.

[12] A.A. Chandio, N. Tziritas, M.S. Chandio, C.-Z. Xu, Energy efficient VM scheduling strategies for HPC workloads in cloud data centers, Sustain. Comput. Inform. Syst. 24 (2019) 100352. https://doi.org/10.1016/j.suscom.2019.100352.

[13] F. Khodayarseresht, E. Soleimani, F. Farahnakian, J. Plosila, Energy and carbon-aware initial virtual machine placement in cloud data centers, Sustain. Comput. Inform. Syst. 39 (2023) 100889. https://doi.org/10.1016/j.suscom.2023.100889.

[14] E. Oner, A.H. Ozer, An energy-aware combinatorial auction-based virtual machine scheduling model and heuristics for green cloud computing, Sustain. Comput. Inform. Syst. 39 (2023) 100913. https://doi.org/10.1016/j.suscom.2023.100913.

[15] D.Q. Mayne, J.B. Rawlings, C.V. Rao, P.O.M. Scokaert, Constrained model predictive control: Stability and optimality, Automatica 36 (6) (2000) 789–814. https://doi.org/10.1016/S0005-1098(99)00214-9.

[16] A. Nemirovski, A. Shapiro, Convex approximations of chance constrained programs, SIAM J. Optim. 17 (4) (2006) 969–996. https://doi.org/10.1137/050622328.

[17] P.K. Donta, T. Varšo, S. Dustdar, Governance and sustainability of distributed continuum systems: a big data approach, J. Big Data 10 (2023) 75. https://doi.org/10.1186/s40537-023-00745-8.

[18] M. Zakarya, L. Gillam, T. Ali, N. Ahmad, N. Iqbal, Sustainable computing across datacenters: A review of enabling models and techniques, Comput. Sci. Rev. 51 (2024) 100613. https://doi.org/10.1016/j.cosrev.2023.100613.

[19] S. Srivastava, N. Bhardwaj, Green Cloud, in: Advances in Environmental Engineering and Green Technologies, IGI Global, 2020. https://doi.org/10.4018/978-1-7998-1971-2.

[20] T.K. Santhosh, R. Krishna, A Decentralized, Energy-efficient cloud architecture for sustainable computing, in: Proc. 2025 Int. Conf. on Emerging Trends in Engineering and Technology, IEEE, 2025.

[21] W. Lin, H. Yang, Z. Lin, A systematic review of green-aware management techniques for sustainable data center, Sustain. Comput. Inform. Syst. 44 (2024) 101009. https://doi.org/10.1016/j.suscom.2024.101009.

[22] S.A. Nesterovich, O.V. Demidovich, The role of distributed computing systems in modern technological ecosystems, in: Proc. Southwest State University Conf. on Computing Systems, 2025.

[23] U.C. De, B. Ray, Green computing for improving the sustainability of data centers: Optimized VM allocation with decentralized peer-to-peer nodes, Int. J. Innov. Comput. Appl. 16 (1) (2025) 1–12.

[24] A. Kumar, V. Sharma, R. Singh, Sustainable Distributed ML Paradigms for Green Edge Intelligence: Analysis and Framework, in: Proc. 2025 6th Int. Conf. on Signal Processing and Communication, IEEE, 2025.

[25] A.P. Sharma, S. Kumar, Sustainable energy efficient workflow classification and scheduling in geo distributed cloud datacenter, Discov. Sustain. 5 (2024) 314. https://doi.org/10.1007/s43621-024-00523-6.

[26] E. Peltonen, J. Harjula, M. Ylianttila, Rethinking Computing Systems in the Era of Climate Crisis: A Call for a Sustainable Computing Continuum, IEEE Internet Comput. 29 (2) (2025) 12–19. https://doi.org/10.1109/MIC.2025.3566642.

[27] G. Abbas, Z. Ahmad, W. Khalid, A. Al-Juiad, A Novel Energy Proficient Computing Framework for Green Computing Using Sustainable Energy Sources, IEEE Access 11 (2023) 78234–78248. https://doi.org/10.1109/ACCESS.2023.3298076.

[28] S. Goudarzi, M. Palaniswami, P. Ciuonzo, Sustainable Edge Node Computing Deployments in Distributed Manufacturing Systems, IEEE Trans. Consum. Electron. 70 (1) (2024) 3150–3161. https://doi.org/10.1109/TCE.2023.3335988.

[29] K.R. Ananth, P. Raja, S. Muthukrishnan, Green IoT Edge Computing Towards Sustainable and Distributed Data Processing, in: Proc. 2023 10th IEEE Uttar Pradesh Section Int. Conf. on Electrical, Electronics and Computer Engineering (UPCON), IEEE, 2023, pp. 1–6.

[30] K. Gai, M. Qiu, L. Tao, Y. Zhu, Resource Management in Sustainable Cyber-Physical Systems Using Heterogeneous Cloud Computing, IEEE Trans. Sustain. Comput. 3 (2) (2018) 60–72. https://doi.org/10.1109/TSUSC.2017.2723954.

[31] L. Wang, J. Zhang, Z. Li, Sustainable Energy-Efficient Multi-Objective Task Processing Based on Edge Computing, IEEE Trans. Netw. Serv. Manag. 22 (1) (2025) 680–694. https://doi.org/10.1109/TNSM.2024.3489012.

[32] M. Akour, A. Al-Hamad, A. Al-Qeisi, Reducing Environmental Impact with Sustainable Serverless Computing, Sustainability 17 (3) (2025) 1124. https://doi.org/10.3390/su17031124.

[33] A. Pazienza, F. Malloci, S. Ferilli, A holistic approach to environmentally sustainable computing, Innov. Syst. Softw. Eng. 20 (2) (2024) 115–134. https://doi.org/10.1007/s11334-023-00546-7.

[34] S. Sarkar, A. Mukherjee, R. Pal, Hierarchical Multi-Agent Framework for Carbon-Efficient Liquid-Cooled Data Center Clusters, arXiv preprint arXiv:2501.05234 (2025).

[35] S.G. Paul, A. Biswas, A. Pramanik, M.S. Islam, S. Bhattacharya, A Comprehensive Review of Green Computing: Past, Present, and Future Research, IEEE Access 11 (2023) 99248–99275. https://doi.org/10.1109/ACCESS.2023.3313511.

[36] A. Radovanovic, R. Koningstein, I. Schneider, B. Chen, A. Duarte, B. Roy, D. Xiao, M. Haridasan, P. Hung, N. Care, S. Talukdar, E. Mullen, K. Smith, M. Cottman, W. Cirne, Carbon-Aware Computing for Datacenters, IEEE Trans. Power Syst. 38 (2) (2023) 1270–1280. https://arxiv.org/abs/2106.11750.

[37] S.K. Mishra, S.P. Singh, Internet of Energy for sustainable smart city energy management: A review, Sustain. Comput. Inform. Syst. 38 (2023) 100881. https://doi.org/10.1016/j.suscom.2023.100881.

[38] B. Asadov, M. Rauber, F. Iorio, Carbon-aware spatio-temporal workload shifting: A systematic literature review, Sustain. Comput. Inform. Syst. 45 (2025) 101020.

[39] A. Hall, P. Marteau, L. Massoulie, Distributionally Robust Optimization for Carbon-Aware Data Center Load Shaping, in: Proc. ACM e-Energy, 2024.

[40] Z. Huang, Q. Liu, R. Ma, T. Li, Improved MPC-based optimal scheduling of hydrogen-based microgrids with security constraints, Appl. Energy 335 (2023) 120734. https://doi.org/10.1016/j.apenergy.2023.120734.

[41] S. Zheng, M. Sun, B. Zhou, Multi-scale MPC scheduling for integrated electricity-thermal-hydrogen energy systems, Energy 285 (2023) 129468.

[42] A. Tilli, M. Bragaglia, C. Conficoni, F. Ferri, Two-layer distributed MPC for thermal management of multiprocessor systems-on-chip, IEEE Trans. Control Syst. Technol. 30 (3) (2022) 1030–1042. https://doi.org/10.1109/TCST.2021.3101604.

[43] G. Perin, L. Badia, F. Paisana, L. Da Silva Cruz, EASE: An MPC-based energy-aware scheduler for vehicular edge networks with renewable energy, in: Proc. IEEE Wireless Communications and Networking Conference (WCNC), 2021, pp. 1–7. https://doi.org/10.1109/WCNC49053.2021.9417390.

[44] G. Perin, G. Cibin, L. Badia, Towards Sustainable Edge Computing Through Renewable Energy Resources and Online, Distributed and Predictive Scheduling, IEEE Trans. Netw. Serv. Manag. 19 (4) (2022) 4996–5010. https://doi.org/10.1109/TNSM.2022.3193236.

[45] B. Kocot, P. Czarnul, J. Proficz, Energy-Aware Scheduling for High-Performance Computing Systems: A Survey, Energies 16 (2023) 890. https://doi.org/10.3390/en16020890.

[46] J. Cao, C. Li, T. Li, Y. Gao, EIS: An energy-aware intelligent scheduling for workflow computing in cloud environment, Sustain. Comput. Inform. Syst. 40 (2023) 100918. https://doi.org/10.1016/j.suscom.2023.100918.

[47] X. Gu, A. Zhang, R. Wang, A. Pang, Joint scheduling of computing resources and green energy for low-carbon edge computing via deep reinforcement learning, Sustain. Comput. Inform. Syst. 40 (2023) 100920.

[48] Y. Wang, L. Zhu, Z. Niu, ERLFC: An energy and emission-aware reinforcement learning task scheduling for federated cloud environments, Sustain. Comput. Inform. Syst. 40 (2023) 100922.

[49] S. Tuli, G. Kecskemeti, N. Jennings, HUNTER: Holistic Scheduling and Resource Management for Sustainable Cloud Networks, J. Syst. Softw. 185 (2022) 111124. https://doi.org/10.1016/j.jss.2021.111124.

[50] S.M. Fard, N. Mehdipour, H. Zhou, Distributed permutation flowshop scheduling with sustainability criteria under real-time control, Sustain. Comput. Inform. Syst. 43 (2024) 100999.

[51] M. Abbasi, A. Dastfan, M. Dehghanian, Parallel simulated annealing for virtual power plant scheduling optimization on HPC infrastructure, Appl. Soft Comput. 150 (2025) 111090.

[52] A. Paul, N. Biswas, S.K. Das, Quantum particle swarm optimization for grid-connected microgrid management, Energy Convers. Manag. 301 (2025) 118018.

[53] J. Ma, Z. Wang, L. Qi, Distributionally robust decarbonizing scheduling for multi-temporal multi-energy microgrids, Appl. Energy 361 (2024) 122933.

[54] W. Dai, H. Li, Z. Chen, Robust multi-energy virtual power plant scheduling under demand and generation uncertainty using Column-and-Constraint Generation, IEEE Trans. Smart Grid 16 (1) (2025) 410–422.